

MAPPING THE MACHINE YOU LIVE ON

COL334 COMPUTER NETWORKS: ASSIGNMENT 1, PARTS A AND B

TEJASVI SHRIVASTAVA (2024CS10582)
YASHVARDHAN SINGH CHAUDHARY (2024CS10300)



A REPORT SUBMITTED FOR
COL334: COMPUTER NETWORKS
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF TECHNOLOGY DELHI

August 2026

Abstract

This report covers Parts A and B of COL334 Assignment 1, in which we measure and document the network infrastructure of the IIT Delhi campus from the vantage point of our own devices, and then turn the same two devices inward and watch what they say on the wire. Working as cartographers, we traced routes to a fixed set of targets and to a further set of campus and external destinations, isolated the internal (10.x.x.x) hops that recur across those traces, and used position, recurrence, and round-trip time to infer the likely role of each hop in the campus topology. We surveyed the wireless layer from three locations, recording every visible SSID, BSSID, band, channel, and signal strength, and used this to determine how many physical access points are actually behind the broadcasts and how the campus's channel plan is enforced. We then ran a set of ping-based delay experiments (fixed-size probes to five targets at increasing distance, a packet-size sweep to isolate transmission delay, and, where data was available, a queue-occupancy comparison between busy and quiet hours) to separate propagation, transmission, and queuing delay in practice. Throughout, we have tried to keep the line between what we directly measured and what we are inferring explicit, and to flag alternative explanations we could not rule out rather than presenting a single tidy story.

Part B reports the traffic-capture half: six ten-minute captures each, over two days and three time bands. Round-trip time across those twelve windows shows a large, sustained shift on the off-campus path that follows the calendar date rather than the hour and is visible on two laptops running different operating systems, while the on-campus evening effect we expected turns out to be weak and inconsistent between the two devices, a negative result we report as one. We also account for what each machine sent unasked, and hunt the anomalies planted in our captures (four in one seed, six in the other), giving each a display filter and the exact frame count it returns, including the two flavours we can show are genuinely absent rather than merely missed. Finally we argue against our own result: which anomaly would be hardest to detect at scale, and what an operator sees that we cannot.

Acknowledgements

We thank the COL334 teaching staff for the assignment design and the shared class map, and IIT Delhi's network infrastructure for tolerating two days of low-rate probing.

Contents

Abstract	ii
Acknowledgements	iii
0 Introduction	1
0.1 Team	1
0.2 Rules of Engagement	1
0.3 Capture Windows	2
0.4 Fixed Targets	2
0.5 Chapter List	2
0.6 A Note on Honesty	3
1 Your Path Out	4
1.1 Device and Access Point	4
1.2 Finding T1	6
1.3 Path to the Boundary	6
1.4 Diagram: Device to Internet	7
1.5 Evidence Tags	9
2 Campus Topology	10
2.1 A2.1: The Router Hunt	10
2.1.1 Internal Topology Graph	12
2.2 A2.2: The Wireless Layer	13
2.2.1 Reading the Survey	13
2.3 A2.3: The Class Map	14
3 The Delay Experiment	16
3.1 A3.1: Where Does the Time Go?	16
3.2 A3.2: Making Transmission Delay Visible	18
3.3 A3.3: Watching a Queue Fill	19

4	Evidence	21
5	Three Findings	30
6	Two Days, Two Devices	32
6.1	Caveats, stated once and relied on throughout	32
6.2	Data	33
6.3	Plots	34
6.4	Q1: Which window was worst for each of us? Same or different?	35
6.4.1	T3 (off campus): the same two windows are worst for both of us, and they share a date	35
6.4.2	How far can we localise it? Not far, and we can show why	35
6.4.3	T2 (on campus): a weak evening effect, and the two devices disagree	36
6.4.4	What the two targets together let us say	37
6.5	Q2: Biggest non-workload consumer of bytes on each device	37
6.5.1	Device 1: 2024CS10300 (Windows 11)	37
6.5.2	Device 2: 2024CS10582 (Ubuntu)	38
6.5.3	Cross-device comparison	39
7	The Hunt	40
7.1	B2.1: How many were planted?	40
7.1.1	Evidence for the two absences on 2024CS10300	41
7.2	B2.2: Anomaly table, 2024CS10300 (SEED_TAG 5A11D1FB)	42
7.3	B2.3: Anomaly table, 2024CS10582 (SEED_TAG 5E857C61)	43
7.4	B2.4: Why these filters and not looser ones	43
7.5	B2.5: Frames returned is a property of the capture, not of the plant . .	45
7.6	B2.6: Required per-teammate block	45
7.7	B2.7: Cross-device observation	46
7.8	B2.8: Confirmation in the re-recorded D2W3	46
8	Argue Against Yourself	48
8.1	Hardest to spot at a million times this traffic: the 11-second beacon (A1)	48
8.2	The innocent case for the beacon	48
8.3	One thing an operator sees that we cannot, and one we see that they cannot	49

List of Tables

1.1	Whois lookups for the first public hops after our gateway.	7
2.1	Observed internal hops and inferred roles. “Traces” is abbreviated to a representative subset with a count; the full per-trace list for each hop is preserved as Evidence E-2.	11
2.2	Wireless survey across three locations (redundant enterprise BSSIDs at Satpura truncated for length; full capture preserved as Evidence E-3). .	13
3.1	Aggregate RTT statistics, 50 packets per target.	16
6.1	T2: <code>www.iitd.ac.in</code> (on campus). Median and p95 RTT in ms, 20 echoes per window per device.	33
6.2	T3: 8.8.8.8 (off campus). Median and p95 RTT in ms. The two windows recorded on 22-08 are set in bold.	33
6.3	The same router, timed twice, minutes apart. Its reported time in the T3 trace tracks the T3 end-to-end RTT rather than any distance to that router.	36
6.4	Per-window totals and the largest non-workload consumer, 2024CS10300.	37
6.5	Per-window totals and the largest non-workload consumer, 2024CS10582.	38
7.1	The six handout flavours tested against both captures. Two are genuinely absent from 2024CS10300’s seed; the evidence for those absences is given below.	41
7.2	Four planted anomalies on 2024CS10300. All rows refer to 2024CS10300_D2_W2.pcapng.	42
7.3	Six planted anomalies on 2024CS10582. All rows refer to 2024CS10582_D2_W2.pcapng.	43
7.4	Four frames, two lookups: the Windows resolver failing over to the secondary server.	45

7.5	Every filter re-run against the re-recorded D2W3 captures. Counts match the reference window exactly.	47
-----	--	----

List of Figures

1.1	Access point nearest Tejasvi's device	5
1.2	Access point nearest Yashvardhan's device	5
1.3	Device-to-Internet path diagram	8
2.1	Internal topology graph	12
3.1	RTT vs. approximate distance	17
3.2	Minimum RTT vs. packet size	18
3.3	Queue saturation, busy vs. quiet hour	19
4.1	Wireless survey rows in the shared class map	22
4.2	Raw router-hunt CSV	23
4.3	Internal router rows in the shared class map	23
4.4	External/CDN router rows in the shared class map	24
4.5	Verification attempt, 10.255.221.1	29
4.6	Verification attempt, 10.255.237.94	29
6.1	PL1: RTT median and p95 per capture window	34
6.2	PL2: RTT p95 by window band	34

Chapter 0

Introduction

Assignment 1 for COL334 (Computer Networks) asks every team to map as much of the IIT Delhi campus network as can honestly be measured from an ordinary student device, without authentication attempts, SNMP, vulnerability scanners, aggressive scan timing, or scanning of anyone else's device. This report covers both halves of the assignment: **Part A: Cartography**, worth 50 marks, and **Part B: The Stakeout**, the traffic-capture half, worth 35 marks. Part A is Chapters [1–5](#); Part B is Chapters [6–8](#).

0.1 Team

This report was prepared jointly by:

- Tejasvi Shrivastava (2024CS10582)
- Yashvardhan Singh Chaudhary (2024CS10300)

0.2 Rules of Engagement

All measurements in this report were taken under the constraints set out in the assignment handout: no authentication attempts, no SNMP, no exploits or vulnerability scanners, scan timing no more aggressive than -T2, and no scanning of devices that might belong to a student or staff member rather than to the infrastructure itself.

0.3 Capture Windows

Part B refers throughout to twelve ten-minute capture windows, six per teammate, labelled $DdWw$ for day d and time band w : W1 is the morning band ($\sim 09:00$ IST), W2 the afternoon band ($\sim 15:00$ IST), and W3 the evening band ($\sim 21:00$ IST). Note that the day labels are collection days, not calendar days: D1W1 and D2W3 were both recorded on 22-08, which turns out to matter in Chapter 6.

0.4 Fixed Targets

Wherever this report refers to “the targets”, it means the five fixed destinations specified in the handout:

- **T1:** our default gateway (found per-device, see Chapter 1)
- **T2:** `www.iitd.ac.in`
- **T3:** `8.8.8.8`
- **T4:** `www.iitb.ac.in` (~ 1100 km)
- **T5:** `www.mit.edu` ($\sim 12,000$ km)

0.5 Chapter List

Chapter 1, Your Path Out. Our own device-to-Internet path: interface, gateway, DNS, and the first hops out of campus, each tagged with evidence.

Chapter 2, Campus Topology. The router hunt (A2.1), the wireless survey (A2.2), and our contribution to the shared class map (A2.3).

Chapter 3, The Delay Experiment. Where RTT time goes (A3.1), making transmission delay visible via a packet-size sweep (A3.2), and watching a queue fill (A3.3).

Chapter 4, Evidence. The numbered evidence items (E-1 onward) that the rest of the report cites.

Chapter 5, Findings. Three specific, checkable findings about the network, each with an alternative explanation we could not rule out.

Chapter 6, Two Days, Two Devices (B1). Twelve ten-minute captures, six per teammate across two days and three time bands, compared for RTT to T2 and T3, and for what each machine sent when we were not asking it to.

Chapter 7, The Hunt (B2). How many anomalies were planted in each of our captures, a display filter and a frame count for every one of them, and why the looser filters give the wrong answer.

Chapter 8, Argue Against Yourself (B3). The anomaly that would be hardest to find at scale, the innocent explanation for it, and what an operator sees that we do not.

0.6 A Note on Honesty

The assignment explicitly rewards the distinction between what we observed directly and what we are inferring, and rewards an honestly reported “no difference” or “couldn’t reach” over a tidied-up positive result. We have tried to keep to that standard throughout: flagging inferred elements, reporting dropped probes as data rather than omitting them, and naming a second explanation wherever our evidence does not fully rule one out.

Chapter 1

Your Path Out

This chapter documents the one part of the network we can see completely: the path from our own device out to the open Internet.

1.1 Device and Access Point

Measurements were taken from both teammates' devices, both in Satpura Hostel but in different rooms. Tejasvi's laptop (Ubuntu) connects over Wi-Fi (interface `wlo1`) from room NB-09; the nearest access point is photographed in Evidence E-1 (Figure [1.1](#)). Yashvardhan connects from room NB-01, also Satpura Hostel; his nearest access point is photographed in Evidence E-2 (Figure [1.2](#)). Both photos carry a handwritten placard giving roll numbers, date, and location.

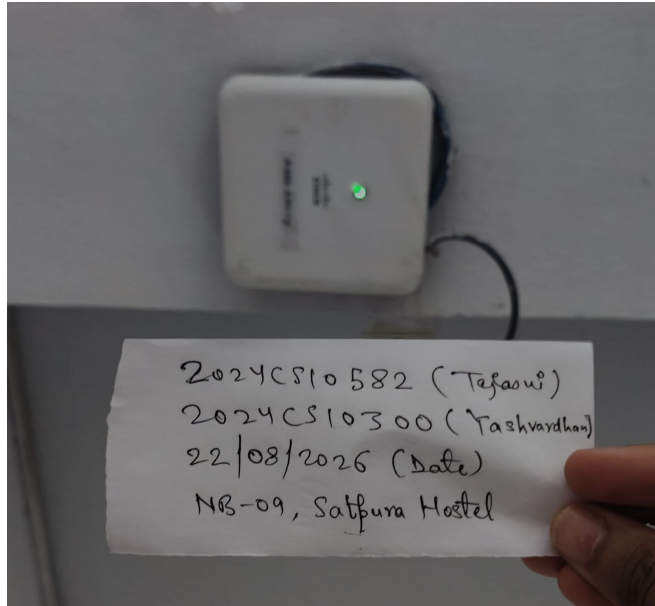


Figure 1.1: The access point/wall unit nearest Tejasvi's device (NB-09) during measurement, with evidence placard. See Evidence E-1.

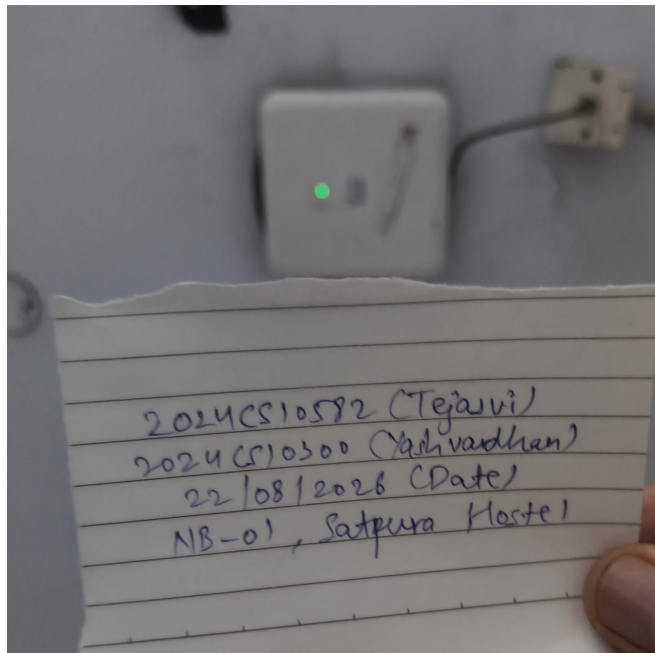


Figure 1.2: The access point/wall unit nearest Yashvardhan's device (NB-01, same hostel), with evidence placard. See Evidence E-2.

1.2 Finding T1

`ip -br a` shows our device’s address as 10.184.6.40/19 on `wlo1`. `ip route | grep default` gives:

```
default via 10.184.0.1 dev wlo1 proto dhcp src 10.184.6.40 metric 600
```

so the DHCP-assigned default gateway is **10.184.0.1**. `resolvectl status` shows our active DNS resolver as **10.7.172.202** (with 10.10.1.4, 10.10.1.5, 10.10.2.2 also configured).

A discrepancy worth flagging. The handout notes that a traceroute to anything outside campus should show the gateway at hop 1 “by definition”, and that if it doesn’t, that’s worth a sentence. In our case it doesn’t: every traceroute we ran (Chapter 2, and the one below) shows hop 1 as **10.184.0.13**, not 10.184.0.1. Both addresses are in the same /19, so our best explanation is that 10.184.0.1 is a virtual/first-hop address (e.g. a shared gateway IP fronting several access switches, possibly via VRRP-style redundancy), while 10.184.0.13 is the specific physical router interface that actually decrements TTL and replies with the ICMP time-exceeded message, i.e. our packets’ first real Layer-3 hop is a different, specific box behind the advertised gateway address. We cannot fully rule out the alternative that 10.184.0.1 simply never responds to traceroute probes (rate-limited or filtered ICMP) while still being the true first hop; we have no way to distinguish “different device” from “same device, doesn’t reply on this address” without further tooling that is out of scope here. For the rest of this report we follow the handout’s implicit intent and use **10.184.0.13** as T1, since that is the address that is consistently and observably our first hop (Evidence E-5; Yashvardhan’s equivalent `ip route`/traceroute output, Evidence E-6, confirms the same address).

1.3 Path to the Boundary

A traceroute to an external destination not previously used elsewhere in this report (www.codechef.com, chosen to avoid double-counting a target already used for T1–T5) gives (Evidence E-7):

```
1  10.184.0.13      4.929 ms  4.856 ms  4.816 ms
2  10.255.109.100   4.761 ms  4.732 ms  4.702 ms
3  10.255.107.3     4.679 ms  4.649 ms  4.622 ms
4  10.119.233.65    4.590 ms  4.560 ms  4.531 ms
5  * * *
6  10.119.234.162   8.483 ms  7.723 ms  7.581 ms
7  * 14.143.159.145 11.753 ms *
8  * * *
9  121.240.255.30   11.581 ms 11.539 ms *
10 104.23.231.30 17.261 ms * 104.23.231.11 9.883 ms
```

11 172.67.75.52 9.713 ms 8.882 ms 8.770 ms

Hop 5 and hop 8 are unresponsive; hop 7 is the first public IP (first boundary crossing); hop 11 is the destination itself. Note that hop 10 returns two different IP addresses on the same line (104.23.231.30 and 104.23.231.11), indicating the upstream network is actively load-balancing our ICMP probes across multiple Cloudflare edge servers via Equal-Cost Multi-Path (ECMP) routing, rather than all three probes hitting the same device.

Hops 1–6 are all internal (10.x.x.x) and match the core/egress path already identified in Table 2.1 (Chapter 2): local gateway, the two core distribution routers, the internal border router, and the external egress gateway. Hop 5 does not respond, which we read as an unresponsive internal hop rather than a gap in the path, since hop 6 replies with a plausible, monotonically-consistent RTT. The first *public* IP appears at hop 7:

Table 1.1: Whois lookups for the first public hops after our gateway.

Hop	IP	Owner (whois)	Role
7	14.143.159.145	Tata Communications (TATACOMM-IN)	Upstream ISP hop
9	121.240.255.30	Tata Communications (TATACOMM-IN, ex-VSNL)	Upstream ISP hop
10	104.23.231.30 / .11	Cloudflare, Inc. (CLOUDFLARENET)	CDN edge

This is where our traffic crosses the IIT Delhi boundary: hop 6 (10.119.234.162, the internal egress gateway) hands off to hop 7, the first Tata Communications address. From there it stays on Tata’s network (hop 9) before landing on a Cloudflare edge node (hop 10) that terminates `www.codechef.com`, consistent with the CDN-edge explanation we gave in Chapter 3 for why T5’s measured minimum RTT undercut its nominal geographic distance: Tata Communications appears to be (at least one of) our commercial upstream(s), and a lot of what looks like “the open Internet” from here is actually a nearby CDN edge rather than the origin server.

1.4 Diagram: Device to Internet

Figure 1.3 draws both teammates’ paths from device to the open Internet, merging at the shared gateway, following the internal core/egress path, crossing the IIT Delhi boundary, and stopping at a dashed line past which everything is inferred rather than directly observed.

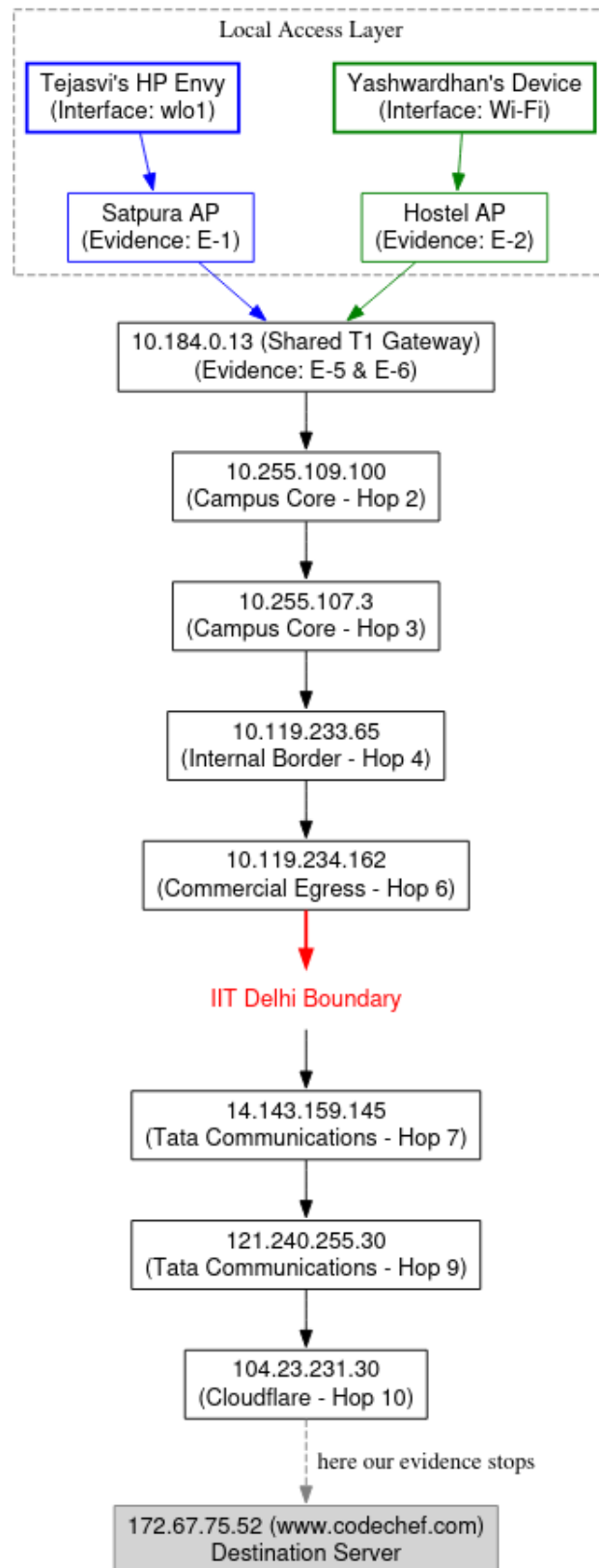


Figure 1.3: Both teammates' device-to-Internet paths, merging at the shared T1 gateway. Evidence tags reference Chapter 4; everything past the dashed line is inferred, not directly observed.

Analytical notes. Two things in this diagram are worth spelling out rather than leaving implicit:

The shared gateway. Both teammates’ independent traceroutes hit **10.184.0.13** at hop 1 (Evidence E-5, E-6), even though they capture from different access points in different rooms (NB-09 and NB-01). Since both rooms are in the same hostel (Satpura), this is a fairly mild result on its own: one first-hop gateway serving one hostel building is the ordinary case, not a surprising one. What it does confirm concretely is that IIT Delhi is not assigning a separate default gateway per room or per AP within a hostel; the access-layer merge happens at (at least) the building level. We cannot rule out the alternative that 10.184.0.13 in fact serves a wider area than just Satpura (multiple hostels sharing one gateway); distinguishing the two would need a third vantage point in a different hostel that still resolves to the same first hop.

The “missing” hops. Hops 5 and 8 in Section 1.3’s traceroute returned * * * (request timeouts). We read this as *not* a routing failure: the surrounding hops (4 and 6; 7 and 9) all reply with plausible, monotonically-increasing RTTs, which would not happen if a hop in between were actually dropping our packets. The more likely explanation is that these specific routers are configured not to generate ICMP time-exceeded replies at all (a common practice for masking internal infrastructure), meaning our packets successfully transited them on the way to the next hop, but the routers themselves declined to identify themselves. The alternative we can’t fully rule out is transient packet loss specific to the one probe we sent per hop; re-running the traceroute with more probes per hop would help distinguish “never replies” from “dropped that particular probe”.

1.5 Evidence Tags

Every element in the final diagram must carry an evidence tag (E-1, E-2, ...) referencing Chapter 4; untagged elements score zero per the assignment’s marking rules.

Chapter 2

Campus Topology

2.1 A2.1: The Router Hunt

Traceroutes were run to at least 10–15 destinations: campus services (`www.iitd.ac.in`, `moodle`, `library`, `webmail`, `cse`, `am`, `chemical`, `design`, `dms`, `textile`, `ee`, `maths`) and external destinations (`1.1.1.1`, `8.8.8.8`, `www.iitb.ac.in`, `www.mit.edu`, and several large commercial sites used purely to sample more of the egress path). Table 2.1 lists every distinct internal (`10.x.x.x`) hop observed, an abbreviated view of which traces it appeared in, its position along the path, its average RTT, and our best guess at its role, reasoned from that position, recurrence, and RTT together.

Table 2.1: Observed internal hops and inferred roles. “Traces” is abbreviated to a representative subset with a count; the full per-trace list for each hop is preserved as Evidence E-2.

Hop IP	Representative (count)	traces	Pos.	Avg RTT (ms)	Best guess at role
10.184.0.13	www.iitd.ac.in, moodle, cse, 8.8.8.8, www.iitb.ac.in, ... (24)		1	4.45	Local gateway (T1) for the hostel area
10.254.236.26	am, webmail, textile, dms, moodle, www.iitd.ac.in, design (7)		2	5.09	Primary aggregation router (campus services)
10.254.236.6	ee, library, chemical, maths (4)		2	4.55	Secondary aggregation router
10.10.17.1	moodle (1)		3	3.62	Department / service subnet gateway
10.254.208.6	cse (1)		2	7.17	Department subnet gateway (CSE)
10.208.20.2	cse (1)		3	7.17	Department subnet gateway (CSE, internal)
10.10.211.213	am (1)		3	8.44	Department / service subnet gateway
10.7.10.101	chemical (1)		3	6.22	Department / service subnet gateway
10.10.17.2	design (1)		4	6.13	Department / service subnet gateway
10.10.211.211	textile (1)		3	7.51	Department / service subnet gateway
10.10.211.216	maths (1)		3	4.96	Department / service subnet gateway
10.255.109.100	whatsapp, amazon, instagram, google, netflix, mit.edu, ... (15)		2	5.01	Campus core distribution router
10.255.107.3	(as above) + www.iitb.ac.in (16)		3	5.39	Campus core distribution router
10.119.233.65	(as above) (16)		4	5.99	Internal border / transit router
10.119.234.162	amazon, ibm, netflix, mit.edu, wikipedia, x, microsoft, reddit, apple (9)		6	8.55	External egress gateway (commercial ISP path)
10.1.207.69	whatsapp, instagram, iitb.ac.in, google, facebook, 8.8.8.8 (6)		5	29.22	Transit router (NKN / peering path)
10.1.200.137	whatsapp, instagram, iitb.ac.in, google, facebook, 8.8.8.8 (6)		6	34.58	Transit router (NKN / peering path)

Note: position/RTT reconstructed from a flattened source table; verify against the original CSV before final submission (likely viva material).

A few observations follow directly from the table. 10.184.0.13 appears at position 1 in every trace we ran, consistent with it being our own default gateway rather than a shared campus device. The pair 10.255.109.100 / 10.255.107.3 / 10.119.233.65 appears in every trace to an external commercial destination but never in traces to `www.iitd.ac.in` or other campus services, which is the signature of a core/egress path that campus-internal traffic never needs to cross. Two hops, 10.1.207.69 and 10.1.200.137, sit at a noticeably higher average RTT (29–35 ms) than the rest of the internal hops (4–9 ms); combined with their appearing only on traces toward external destinations and not toward `www.iitb.ac.in` exclusively, we read them as the peering/transit hops just before or on IIT Delhi’s connection to the National Knowledge Network (NKN) or a commercial upstream, rather than anything internal to a single building. The department gateways (10.10.17.x, 10.10.211.x, 10.208.20.2, 10.7.10.101, 10.254.208.6) each appeared in exactly one trace, which is expected: each department’s traceroute only crosses its own subnet gateway.

2.1.1 Internal Topology Graph

Figure 2.1 draws the internal topology as a graph, with routers as nodes and observed adjacencies (which hop directly followed which, across all traces) as edges.

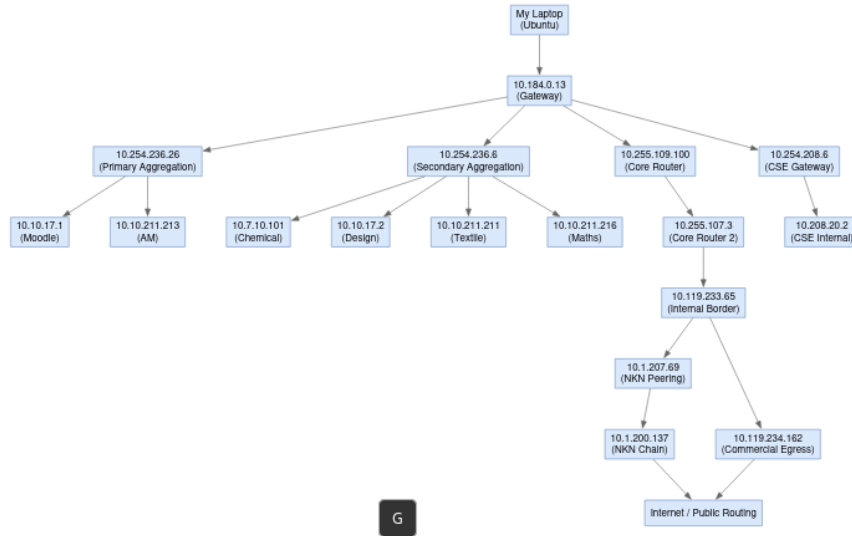


Figure 2.1: Internal topology graph built from observed adjacencies in Table 2.1. Edges represent a hop directly following another in at least one trace; nothing past the last labelled node was directly observed.

2.2 A2.2: The Wireless Layer

A Wi-Fi survey was run from three locations on campus using `nmcli`, recording every broadcast SSID together with its BSSID, band, channel, and signal strength.

Table 2.2: Wireless survey across three locations (redundant enterprise BSSIDs at Satpura truncated for length; full capture preserved as Evidence E-3).

Location	BSSID	SSID	Band	Ch.	Sig.
Satpura	4E:BC:FE:A5:A4:A4	realme P2 Pro 5G	2.4 GHz	6	97
Satpura	E4:4E:2D:0C:82:61	eduroam	2.4 GHz	1	74
Satpura	E4:4E:2D:0C:82:60	IITD_WIFI	2.4 GHz	1	74
Satpura	E4:4E:2D:0C:82:64	IITD_Secure_GUEST	2.4 GHz	1	74
Satpura	E4:4E:2D:0C:82:63	(Hidden)	2.4 GHz	1	74
Satpura	E4:4E:2D:0C:82:6E	eduroam	5 GHz	44	70
Satpura	E4:4E:2D:0C:82:6F	IITD_WIFI	5 GHz	44	69
Satpura	E4:4E:2D:0C:8B:E1	eduroam	2.4 GHz	6	59
Satpura	A4:B4:39:40:C3:61	eduroam	2.4 GHz	11	59
Satpura	A4:B4:39:33:7E:A4	IITD_Secure_GUEST	2.4 GHz	11	40
Satpura	88:9C:AD:5A:43:41	eduroam	2.4 GHz	11	29
Lecture Hall	54:8A:BA:DD:7A:C1	IITD_Secure_GUEST	2.4 GHz	6	97
Lecture Hall	54:8A:BA:DD:7A:C4	IITD_WIFI	2.4 GHz	6	87
Lecture Hall	54:8A:BA:DD:7A:C3	IITD_IOT	2.4 GHz	6	87
Lecture Hall	54:8A:BA:DD:7A:CE	IITD_Secure_GUEST	5 GHz	36	82
Lecture Hall	04:5F:B9:2F:24:C1	IITD_Secure_GUEST	2.4 GHz	1	69
Lecture Hall	70:BD:96:6E:75:42	eduroam	2.4 GHz	11	65
Lecture Hall	38:7A:CC:6D:56:A9	3E4C54F42B90	5 GHz	44	59
Lecture Hall	78:22:88:40:9F:42	454B10749644	5 GHz	44	44
Lecture Hall	04:5F:B9:2F:24:CB	IITD_WIFI	5 GHz	116	42
Library	70:DF:2F:04:98:62	eduroam	2.4 GHz	1	72
Library	70:DF:2F:04:98:63	IITD_IOT	2.4 GHz	1	72
Library	5C:A6:2D:44:E7:E2	eduroam	2.4 GHz	11	62
Library	5C:A6:2D:44:E7:ED	eduroam	5 GHz	128	56
Library	70:DF:2F:04:98:6C	IITD_IOT	5 GHz	149	52
Library	B8:B7:DB:E6:BD:B5	26582626	2.4 GHz	5	42
Library	BE:00:19:A0:0C:FA	rjnish	2.4 GHz	6	40
Library	5C:A6:2D:6F:F8:E1	IITD_Secure_GUEST	2.4 GHz	1	39

2.2.1 Reading the Survey

How many physical APs. SSIDs are not access points: a single radio commonly broadcasts several SSIDs from one MAC block (multiple BSSID / MBSSID), differing only in the last hex digit. Grouping BSSIDs by their shared prefix rather than counting SSID rows gives 5 distinct physical enterprise APs in Satpura, 3 in the Lecture Hall,

and 4 in the Library, noticeably fewer than the raw broadcast count.

Global vs. localised SSIDs. `eduroam`, `IITD_WIFI`, and `IITD_Secure_GUEST` appear in all three locations and are evidently provisioned campus-wide. `IITD_IOT` appears in both academic locations (Lecture Hall, Library) but not once in the Satpura data, which reads as a deliberate choice to keep IoT infrastructure out of residential buildings rather than a coverage gap, though we cannot rule out that our single Satpura vantage point simply missed an `IITD_IOT` radio elsewhere in the hostel. Personal hotspots (`realme P2 Pro 5G`, `rjnish`, the two unnamed hex-string SSIDs in the Lecture Hall) are, as expected, seen at exactly one location each and nowhere else.

Channel plan. Across all three surveys, zero networks were observed on 2.4 GHz channels 3, 4, or 8: every enterprise AP sits on 1, 6, or 11, the three mutually non-overlapping channels. This is consistent with the campus enforcing a fixed channel plan (RRM) on managed infrastructure. The one exception is the student hotspot 26582626 in the Library, broadcasting on channel 5, which does create adjacent-channel interference with neighbouring channel-1 and channel-6 networks, but since it is a personal device we cannot scan or otherwise investigate further under the assignment’s rules.

2.3 A2.3: The Class Map

Our router-hunt and wireless-survey data (Sections 2.1–2.2) were added to the shared `COL334_A1_Campus_Map.xlsx`, and we independently verified two rows submitted by another team.

Contributions. We contributed 33 previously-undocumented infrastructure IPs to the shared map (Evidence E-3, E-4, which already cover the underlying wireless-survey and router-hunt data this was built from): the internal aggregation-layer routers already listed in Table 2.1 (e.g. `10.254.236.x`), plus a further set of external transit and CDN-edge addresses seen only on paths to non-campus destinations, including NKN peering (`10.1.200.137`), Meta’s peering/transit edge, and various CDN edges for `mit.edu` and `google.com`, among others. These were left out of Table 2.1 itself to keep it focused on the destinations this report discusses in depth, but they are now part of the class-wide picture.

Verification. We selected two rows submitted by another team (2024CS50261+2024CS51073) to check independently from our own vantage point:

- Row 1092: `10.255.221.1`, claimed at hop 11.
- Row 1096: `10.255.237.94`, claimed at hop 7.

Both came back **couldn't reach**: a fresh `tracert -n` and `ping -c 10` to each IP, run directly from the Satpura access layer, returned 100% timeouts/packet loss across all 30 traceroute hops and all 10 pings (Evidence E-15). We read this as more likely an architectural access-control choice than a sign the other team's row is wrong: these could be core routers that simply do not answer unsolicited ICMP originating from student-subnet IP space at all. That would be distinct from the hop 5/hop 8 case in Chapter 1, which only suppressed TTL-exceeded replies while still forwarding transit traffic; here nothing came back at all, including to packets that were never expected to traverse these IPs as intermediate hops. The other explanation we can't rule out is more mundane: the other team's claimed hop-position or IP could simply be a typo or a stale reading from when that address briefly appeared in one of their traces, in which case "couldn't reach" would reflect an error in the source row rather than anything about the router's configuration. We have no way to distinguish the two without asking the submitting team what they saw at the time.

Chapter 3

The Delay Experiment

3.1 A3.1: Where Does the Time Go?

Each of T1–T5 was pinged 50 times; Table 3.1 records the minimum, average, and maximum RTT observed for each. Distances are rough straight-line estimates (T2/T4 from map distance to the named campus, T3/T5 from public IP geolocation, i.e. the resolved IP’s registered location) and are not independently verified against ground truth.

Table 3.1: Aggregate RTT statistics, 50 packets per target.

Target (role)	Resolved IP	Est. distance	Min RTT	Avg RTT	Max RTT
T1 (local gate-way)	10.184.0.13	<0.1 km	3.904	8.891	96.290
T2 (campus target)	www.iitd.ac.in	~2.0 km	3.315	7.489	14.110
T3 (regional DNS)	8.8.8.8	~20.0 km	42.824	49.859	119.830
T4 (IIT Bombay)	www.iitb.ac.in	~1100 km	N/A: 100% packet loss		
T5 (global target)	www.mit.edu	nominal 12,000 km	35.596	51.145	247.982

All RTT values in ms.

T4 returned 100% packet loss across all 50 probes. We read this as ICMP being firewalled at or near `www.iitb.ac.in` rather than a routing failure, since traceroutes toward the same destination (Chapter 2) do get responses from intermediate hops, which would not happen if the path itself were broken.

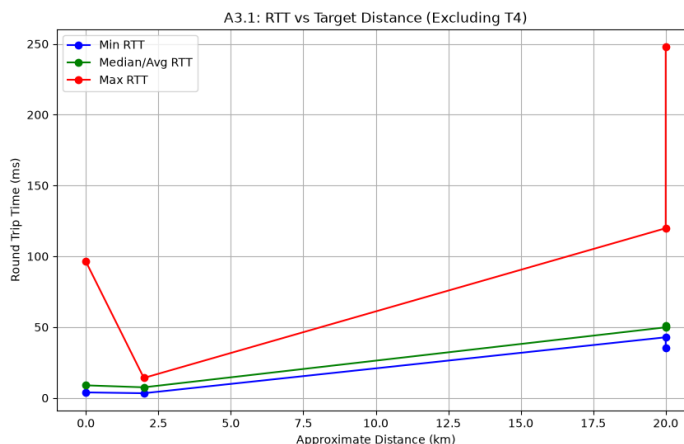


Figure 3.1: Minimum, median/average, and maximum RTT against approximate distance for T1, T2, T3, T5 (T4 excluded, no response).

Propagation delay vs. measured minimum. At 2×10^8 m/s, T2’s ~ 2 km distance implies a propagation delay of about 0.02 ms, negligible next to the measured 3.3 ms minimum, so for a target this close essentially all of the RTT floor is transmission and processing delay, not propagation. T5’s 12,000 km distance implies a propagation floor of about 120 ms round-trip, yet the measured minimum was 35.6 ms, well under that floor. The straightforward reading is that our traffic did not actually travel the full nominal distance: www.mit.edu is very likely served to us from a CDN edge node much closer than Massachusetts. An alternative we cannot fully rule out is that the geolocation-based distance estimate itself is simply wrong (IP geolocation for large anycast/CDN-fronted services is frequently inaccurate), in which case the “true” distance to whatever server actually answered us is just smaller than 12,000 km rather than the packet having taken an implausible shortcut. Either way, the number confirms we are not talking to a server in Cambridge, MA.

Min-to-max/average gap is queuing delay. Propagation and transmission delay for a given path and packet size are effectively fixed, so the spread between the minimum and the average/maximum RTT for the same target is attributable to queuing delay: time spent sitting in router buffers behind other traffic. This is why it shows up as *variation* rather than a constant offset: queue occupancy depends on how much cross-traffic happens to be competing for the same link at the moment our packet arrives, which changes probe to probe. T3 shows the widest such gap (42.8 ms min vs. 119.8 ms max), consistent with a longer, shared path through more congested infrastructure than the one-hop path to T1.

T1’s non-zero RTT. T1 is one hop away, yet its minimum RTT is 3.9 ms, not 0.

Over Wi-Fi, this floor is built from MAC-layer overhead we do not see in the ICMP timestamp: CSMA/CA clear-channel assessment before the medium can be used, the mandatory DIFS/backoff interval, and (in both directions) the transmission delay of putting the frame on and off the physical layer, plus whatever small forwarding/processing delay the gateway itself adds. None of this is propagation delay: at <0.1 km, propagation is on the order of nanoseconds.

3.2 A3.2: Making Transmission Delay Visible

T1 and T3 were pinged at packet sizes 64, 256, 512, 1024, and 1400 bytes (30 packets each); the minimum RTT at each size was plotted against packet size and fit with a straight line for each target (Figure 3.2).

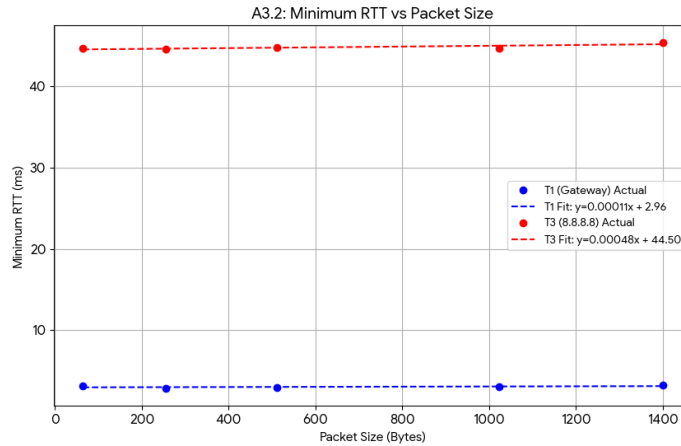


Figure 3.2: Minimum RTT against packet size for T1 and T3, with linear fits.

Fitted lines: T1, $y = 0.00011x + 2.96$; T3, $y = 0.00048x + 44.50$ (RTT in ms, x in bytes).

The intercept. The y-intercept is the extrapolated RTT for a 0-byte payload, i.e. everything that is *not* transmission delay for the payload itself: propagation delay, router/host processing, and (for T1) Wi-Fi contention overhead. T3's much larger intercept (44.5 ms vs. 2.96 ms) reflects the extra propagation and per-hop processing accumulated over the longer, multi-hop path to 8.8.8.8, versus the single Wi-Fi hop to T1.

The slope. The slope is transmission delay per byte, L/R , and since a multi-hop path uses store-and-forward switching, the effective slope is the *sum* of $1/R_i$ over every link on the path, not just the first one. T1's slope (0.00011 ms/byte) is close to flat

because one Wi-Fi hop’s link rate is high enough that transmission delay for a payload in this size range barely moves the RTT. T3’s slope (0.00048 ms/byte) is roughly four times steeper, which is consistent with the packet crossing several additional links (campus core, egress, upstream) each contributing their own L/R_i term, rather than transmission delay being determined by a single bottleneck link. We note that both slopes are small enough, and the underlying ping resolution coarse enough, that we would not want to over-read the precise ratio between them: the qualitative story (T3 accumulates more transmission delay across more hops) is what the data supports; the exact “4×” figure is more fragile.

3.3 A3.3: Watching a Queue Fill

T3 (8.8.8.8) was pinged continuously for 5 minutes (300 packets) at a quiet hour (20-08-2026, 05:40) and again at a busy hour (22-08-2026, 15:11). Figure 3.3 plots RTT against sequence number for both.

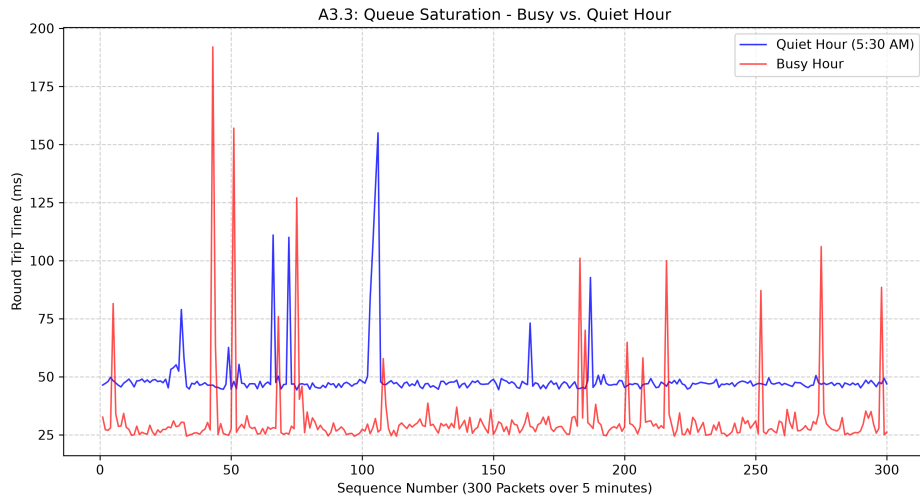


Figure 3.3: RTT vs. packet sequence number, 300 pings to T3, quiet hour (20-08-2026, 05:40) vs. busy hour (22-08-2026, 15:11). The plot legend rounds the quiet-hour start to 5:30 AM.

Which delay component moved, which didn’t. The spikes in both traces (and there are noticeably more of them in the busy-hour trace) are queuing delay: propagation and transmission delay for a fixed-size packet on a fixed path do not vary probe-to-probe, so any variation has to be time spent sitting in a router buffer behind other traffic.

What we did not expect is that the two traces sit on different *baselines*: quiet hour

settles around 48 ms, busy hour around 27 ms. Baseline (the floor propagation + transmission + fixed processing delay would produce with an empty queue) should not depend on time of day, so a 20 ms shift in the floor itself is not something queuing alone explains. The most likely account is that the two captures did not use the same path: something changed the route to 8.8.8.8 between the two windows, most plausibly Google’s anycast routing handing us off to a different, physically closer edge for one of the two captures, though we cannot rule out a more mundane explanation on our end, such as the device having roamed to a different AP or uplink between captures. We do not have a way to confirm which, from ping data alone (that would need a traceroute run in each window, which we did not capture at the time), but whichever it is, it means this pair of captures is confounded for a pure “same path, different hour” queuing comparison, and the baseline shift itself is arguably the more interesting result of the two.

What the shape tells us. Looking past the baseline shift, the two traces do still differ in shape the way the assignment expects: the quiet-hour trace is flat with only a couple of isolated spikes (max ~ 155 ms), consistent with a queue that is usually empty. The busy-hour trace is persistently spiky: frequent, sharp excursions up to ~ 190 ms that return to baseline almost immediately rather than decaying gradually. That shape (sharp in, sharp out, rather than a sustained elevated floor) reads as bursty cross-traffic, short bursts that fill a buffer momentarily and drain quickly once the burst ends, rather than the queue being continuously close to full. So while we can’t cleanly isolate “busy hour vs. quiet hour on the same path”, what we can say is that the busy-hour capture, whatever path it took, shows a network under frequent transient load rather than sustained congestion.

Chapter 4

Evidence

The assignment requires 10–15 numbered evidence items, E-1 onward with no gaps, each stating who took it, when, where, the artefact itself, and which map element/table row/plot it supports.

E-1: photo

Who: Tejasvi Shrivastava (2024CS10582) When: 22-08-2026, 03:30 Where: NB-09, Satpura Hostel

Artefact: Figure 1.1, photo of the access point nearest Tejasvi’s device, with handwritten placard (roll numbers, date, location).

Supports: Chapter 1, Section “Device and Access Point”; Figure 1.3.

E-2: photo

Who: Yashvardhan Singh Chaudhary (2024CS10300) When: 22-08-2026, 18:59 Where: NB-01, Satpura Hostel

Artefact: Figure 1.2, photo of the access point nearest Yashvardhan’s device, with handwritten placard (roll numbers, date, location).

Supports: Chapter 1, Section “Device and Access Point”; Figure 1.3.

E-3: survey

Who: Tejasvi Shrivastava (2024CS10582) When: 22-08-2026, 05:27 (Satpura), with Lecture Hall and Library scans following the same session Where: Satpura Hostel, Lecture Hall, Library

Artefact: verbatim `nmcli -f BSSID,SSID,FREQ,CHAN,SIGNAL dev wifi` output, all three locations, untruncated (reproduced below; note the ~5h27m gap the shell itself recorded between the Satpura and Lecture Hall scans).

```
~  
> nmcli -f BSSID,SSID,FREQ,CHAN,SIGNAL dev wifi
```

```

BSSID          SSID          FREQ      CHAN  SIGNAL
4E:BC:FE:A5:A4:A4  realme P2 Pro 5G  2437 MHz  6      97
E4:4E:2D:0C:82:61  eduroam          2412 MHz  1      74
E4:4E:2D:0C:82:60  IITD_WIFI        2412 MHz  1      74
... (22 BSSIDs total; full list in evidences/e2.txt)

~ took 5h27m2s
> nmcli -f BSSID,SSID,FREQ,CHAN,SIGNAL dev wifi
54:8A:BA:DD:7A:C1  IITD_Secure_GUEST 2437 MHz  6      97
... (21 BSSIDs total; full list in evidences/e2.txt)

~ took 6s
> nmcli -f BSSID,SSID,FREQ,CHAN,SIGNAL dev wifi
70:DF:2F:04:98:62  eduroam          2412 MHz  1      72
... (21 BSSIDs total; full list in evidences/e2.txt)

```

Supports: Chapter 2, Section A2.2 wireless survey table. (Note: the raw scan carries more BSSIDs per site than Table 2.2, since each physical AP was broadcasting the full MBSSID set on both bands here; Table 2.2 already reflects this in its “how many physical APs” count.) This same survey data was subsequently entered into the shared class map (Figure 4.1), so this item also supports Section A2.3’s contribution count.

EVIDENCE	TIME	SSID	BSSID	FREQ	CHAN	SIGNAL	LOCATION	DATE
9176	2024CS10582+2024CS10300	realme P2 Pro 5G	4E:BC:FE:A5:A4:A4	2.4 GHz	6	-51	Satpura Hostel	19-08-2026
9177	2024CS10582+2024CS10300	eduroam	E4:4E:2D:0C:82:61	2.4 GHz	1	-63	Satpura Hostel	19-08-2026
9178	2024CS10582+2024CS10300	IITD_WIFI	E4:4E:2D:0C:82:60	2.4 GHz	1	-63	Satpura Hostel	19-08-2026
9179	2024CS10582+2024CS10300	IITD_Secure_GUEST	E4:4E:2D:0C:82:64	2.4 GHz	1	-63	Satpura Hostel	19-08-2026
9180	2024CS10582+2024CS10300	(Hidden)	E4:4E:2D:0C:82:63	2.4 GHz	1	-63	Satpura Hostel	19-08-2026
9181	2024CS10582+2024CS10300	eduroam	E4:4E:2D:0C:82:6E	5 GHz	44	-65	Satpura Hostel	19-08-2026
9182	2024CS10582+2024CS10300	IITD_WIFI	E4:4E:2D:0C:82:6F	5 GHz	44	-65	Satpura Hostel	19-08-2026
9183	2024CS10582+2024CS10300	IITD_Secure_GUEST	E4:4E:2D:0C:82:6B	5 GHz	44	-65	Satpura Hostel	19-08-2026
9184	2024CS10582+2024CS10300	eduroam	E4:4E:2D:0C:8B:E1	2.4 GHz	6	-70	Satpura Hostel	19-08-2026
9185	2024CS10582+2024CS10300	IITD_WIFI	E4:4E:2D:0C:8B:E0	2.4 GHz	6	-70	Satpura Hostel	19-08-2026
9186	2024CS10582+2024CS10300	IITD_Secure_GUEST	E4:4E:2D:0C:8B:E4	2.4 GHz	6	-70	Satpura Hostel	19-08-2026
9187	2024CS10582+2024CS10300	eduroam	A4:84:39:40:C3:61	2.4 GHz	11	-70	Satpura Hostel	19-08-2026
9188	2024CS10582+2024CS10300	IITD_WIFI	A4:84:39:40:C3:60	2.4 GHz	11	-71	Satpura Hostel	19-08-2026
9189	2024CS10582+2024CS10300	IITD_Secure_GUEST	A4:84:39:40:C3:64	2.4 GHz	11	-71	Satpura Hostel	19-08-2026
9190	2024CS10582+2024CS10300	(Hidden)	A4:84:39:40:C3:63	2.4 GHz	11	-71	Satpura Hostel	19-08-2026
9191	2024CS10582+2024CS10300	IITD_Secure_GUEST	A4:84:39:33:7E:A4	2.4 GHz	11	-80	Satpura Hostel	19-08-2026
9192	2024CS10582+2024CS10300	(Hidden)	A4:84:39:33:7E:A3	2.4 GHz	11	-80	Satpura Hostel	19-08-2026
9193	2024CS10582+2024CS10300	IITD_WIFI	A4:84:39:33:7E:A0	2.4 GHz	11	-80	Satpura Hostel	19-08-2026
9194	2024CS10582+2024CS10300	eduroam	A4:84:39:33:7E:A1	2.4 GHz	11	-81	Satpura Hostel	19-08-2026
9195	2024CS10582+2024CS10300	eduroam	88:9C:AD:5A:43:41	2.4 GHz	11	-85	Satpura Hostel	19-08-2026
9196	2024CS10582+2024CS10300	IITD_Secure_GUEST	88:9C:AD:5A:43:44	2.4 GHz	11	-86	Satpura Hostel	19-08-2026
9197	2024CS10582+2024CS10300	IITD_WIFI	88:9C:AD:5A:43:40	2.4 GHz	11	-86	Satpura Hostel	19-08-2026
9198	2024CS10582+2024CS10300	IITD_Secure_GUEST	54:8A:BA:DD:7A:C1	2.4 GHz	6	-51	Lecture Hall	19-08-2026
9199	2024CS10582+2024CS10300	eduroam	54:8A:BA:DD:7A:C2	2.4 GHz	6	-54	Lecture Hall	19-08-2026
9200	2024CS10582+2024CS10300	IITD_WIFI	54:8A:BA:DD:7A:C4	2.4 GHz	6	-56	Lecture Hall	19-08-2026
9201	2024CS10582+2024CS10300	IITD_IOT	54:8A:BA:DD:7A:C3	2.4 GHz	6	-56	Lecture Hall	19-08-2026
9202	2024CS10582+2024CS10300	IITD_Secure_GUEST	54:8A:BA:DD:7A:CE	5 GHz	36	-59	Lecture Hall	19-08-2026
9203	2024CS10582+2024CS10300	IITD_WIFI	54:8A:BA:DD:7A:CB	5 GHz	36	-60	Lecture Hall	19-08-2026
9204	2024CS10582+2024CS10300	IITD_IOT	54:8A:BA:DD:7A:CC	5 GHz	36	-60	Lecture Hall	19-08-2026
9205	2024CS10582+2024CS10300	IITD_Secure_GUEST	04:5F:B9:2F:24:C1	2.4 GHz	1	-65	Lecture Hall	19-08-2026
9206	2024CS10582+2024CS10300	IITD_WIFI	04:5F:B9:2F:24:C4	2.4 GHz	1	-65	Lecture Hall	19-08-2026
9207	2024CS10582+2024CS10300	IITD_IOT	04:5F:B9:2F:24:C3	2.4 GHz	1	-65	Lecture Hall	19-08-2026
9208	2024CS10582+2024CS10300	eduroam	54:8A:BA:DD:7A:CD	5 GHz	36	-66	Lecture Hall	19-08-2026
9209	2024CS10582+2024CS10300	eduroam	04:5F:B9:2F:24:C2	2.4 GHz	1	-66	Lecture Hall	19-08-2026
9210	2024CS10582+2024CS10300	eduroam	70:BD:96:6E:75:42	2.4 GHz	11	-67	Lecture Hall	19-08-2026
9211	2024CS10582+2024CS10300	IITD_Secure_GUEST	70:BD:96:6E:75:41	2.4 GHz	11	-68	Lecture Hall	19-08-2026
9212	2024CS10582+2024CS10300	IITD_WIFI	70:BD:96:6E:75:44	2.4 GHz	11	-68	Lecture Hall	19-08-2026
9213	2024CS10582+2024CS10300	IITD_IOT	70:BD:96:6E:75:43	2.4 GHz	11	-68	Lecture Hall	19-08-2026
9214	2024CS10582+2024CS10300	3EAC5AF42B80	38:7A:CC:8D:56:A9	5 GHz	44	-70	Lecture Hall	19-08-2026
9215	2024CS10582+2024CS10300	454B10749644	78:22:88:40:9F:42	5 GHz	44	-78	Lecture Hall	19-08-2026
9216	2024CS10582+2024CS10300	IITD_WIFI	04:5F:B9:2F:24:CB	5 GHz	116	-79	Lecture Hall	19-08-2026

Figure 4.1: Our wireless-survey rows (Evidence E-3) entered into COL334.A1.Campus_Map.xlsx, spanning all three surveyed locations.

E-4: spreadsheet

Who: Tejasvi Shrivastava (2024CS10582) When: 19-08-2026, 19:30 Where: NB-09, Satpura Hostel

Artefact: Figure 4.2: A2_router.table.csv, the raw per-trace hop data underlying Table 2.1.

```

A2_router_table.csv |> data
1 Hop IP Appears in which traces, Position in each trace, Avg RTT (ms), Best guess at its role
2 10.184.0.13, "library.iitd.ac.in, webmail.iitd.ac.in, www.google.com, www.mit.edu, maths.iitd.ac.in, www.facebook.com, chemical.iitd.ac.in, am.iitd.ac.in, www.x.com, www.microsoft.com, www.
3 10.254.236.26, "am.iitd.ac.in, webmail.iitd.ac.in, textile.iitd.ac.in, dms.iitd.ac.in, moodle.iitd.ac.in, www.iitd.ac.in, design.iitd.ac.in", "www.iitd.ac.in(2), moodle.iitd.ac.in(2), webmai
4 10.10.17.1, moodle.iitd.ac.in, moodle.iitd.ac.in(3), 3.62,
5 10.254.236.6, "ee.iitd.ac.in, library.iitd.ac.in, chemical.iitd.ac.in, maths.iitd.ac.in", "library.iitd.ac.in(2), chemical.iitd.ac.in(2), ee.iitd.ac.in(2), maths.iitd.ac.in(2)", 4.55,
6 10.254.208.6, cse.iitd.ac.in, cse.iitd.ac.in(2), 7.17,
7 10.208.20.2, cse.iitd.ac.in, cse.iitd.ac.in(3), 7.17,
8 10.10.211.213, am.iitd.ac.in, am.iitd.ac.in(3), 8.44,
9 10.7.10.101, chemical.iitd.ac.in, chemical.iitd.ac.in(3), 6.22,
10 10.10.17.2, design.iitd.ac.in, design.iitd.ac.in(4), 6.13,
11 10.10.211.211, textile.iitd.ac.in, textile.iitd.ac.in(3), 7.51,
12 10.10.211.216, maths.iitd.ac.in, maths.iitd.ac.in(3), 4.96,
13 10.255.109.109, "www.whatsapp.com, www.amazon.com, www.instagram.com, www.apple.com, www.ibm.com, www.google.com, www.netflix.com, www.mit.edu, www.wikipedia.org, www.x.com, www.facebook.co
14 10.255.107.3, "www.whatsapp.com, www.amazon.com, www.instagram.com, www.apple.com, www.iitb.ac.in, www.ibm.com, www.google.com, www.netflix.com, www.mit.edu, www.wikipedia.org, www.x.com, w
15 10.119.233.65, "www.whatsapp.com, www.amazon.com, www.instagram.com, www.apple.com, www.iitb.ac.in, www.ibm.com, www.google.com, www.netflix.com, www.mit.edu, www.wikipedia.org, www.x.com, w
16 10.1.207.69, "www.whatsapp.com, www.instagram.com, www.iitb.ac.in, www.google.com, www.facebook.com, 8.8.8.8", "8.8.8.8(5), www.iitb.ac.in(5), www.google.com(5), www.instagram.com(5), www.f
17 10.1.208.137, "www.whatsapp.com, www.instagram.com, www.iitb.ac.in, www.google.com, www.facebook.com, 8.8.8.8", "8.8.8.8(6), www.iitb.ac.in(6), www.google.com(6), www.instagram.com(6), www.f
18 10.255.238.122, 8.8.8.8, 8.8.8.8(7), 46.33,
19 10.152.7.214, "www.whatsapp.com, 8.8.8.8, www.google.com", "8.8.8.8(8), www.google.com(8), www.whatsapp.com(8)", 44.6,
20 10.1.207.122, www.iitb.ac.in, www.iitb.ac.in(6), 14.49,
21 10.120.123.118, www.iitb.ac.in, www.iitb.ac.in(7), 14.32,
22 10.1.207.121, www.iitb.ac.in, www.iitb.ac.in(8), 43.07,
23 10.255.238.254, "www.iitb.ac.in, www.google.com", "www.iitb.ac.in(10), www.google.com(7)", 44.47,
24 10.119.249.49, www.instagram.com, www.instagram.com(7), 12.77,
25 10.119.249.50, www.instagram.com, www.instagram.com(8), 6.11,
26 10.10.10.1, www.iitb.ac.in, www.iitb.ac.in(13), 51.74,
27 10.119.234.162, "www.amazon.com, www.ibm.com, www.netflix.com, www.mit.edu, www.wikipedia.org, www.x.com, www.microsoft.com, www.reddit.com, www.apple.com, www.pornhub.com", "www.mit.edu(6),
28 10.255.239.30, www.instagram.com, www.instagram.com(7), 12.77,
29 10.255.222.229, www.instagram.com, www.instagram.com(8), 6.11,
30 10.255.222.1, www.instagram.com, www.instagram.com(9), 12.72,
31 10.24.250.102, www.facebook.com, www.facebook.com(7), 12.91,
32 10.120.71.174, www.facebook.com, www.facebook.com(8), 12.89,
33 10.255.221.1, www.facebook.com, www.facebook.com(9), 12.83,
34 10.255.237.94, www.whatsapp.com, www.whatsapp.com(7), 48.58,
35 172.66.0.227, www.x.com, www.x.com(14), 42.28,

```

Figure 4.2: Raw A2.router_table.csv data underlying Table 2.1. More hops appear here than in the table; the extras are single-appearance transit nodes on paths to sites not otherwise discussed (e.g. facebook.com, x.com) and were left out of Table 2.1 to keep it to the destinations this report actually analyses.

This same router data was entered into the shared class map as our A2.3 contribution, split between internal (Figure 4.3) and external/CDN (Figure 4.4) hops:

Trace ID	Source IP	Destination IP	RTT (ms)	Role	Path	Found in	Example target
2031	202AC810582-202AC810300	10.184.0.13	1.28 of 29	4.45 Sarpura Hostel	Local Gateway (T1)	High	Found in 28 traces. Example target: library.iitd.ac.in
2032	202AC810582-202AC810300	10.254.236.26	2.7 of 29	5.09 Sarpura Hostel	Primary Aggregation	High	Found in 7 traces. Example target: am.iitd.ac.in
2033	202AC810582-202AC810300	10.10.17.1	3.1 of 29	3.62 Sarpura Hostel	Department Subnet Gateway	Medium	Found in 1 traces. Example target: moodle.iitd.ac.in
2034	202AC810582-202AC810300	10.254.236.6	2.4 of 29	4.55 Sarpura Hostel	Secondary Aggregation	High	Found in 6 traces. Example target: ee.iitd.ac.in
2035	202AC810582-202AC810300	10.254.208.6	2.1 of 29	7.17 Sarpura Hostel	CSE Subnet Gateway	Medium	Found in 1 traces. Example target: cse.iitd.ac.in
2036	202AC810582-202AC810300	10.208.20.2	3.1 of 29	7.17 Sarpura Hostel	CSE Subnet Gateway	Medium	Found in 1 traces. Example target: cse.iitd.ac.in
2037	202AC810582-202AC810300	10.10.211.213	3.1 of 29	8.44 Sarpura Hostel	Department Subnet Gateway	Medium	Found in 1 traces. Example target: am.iitd.ac.in
2038	202AC810582-202AC810300	10.10.10.1	3.1 of 29	6.22 Sarpura Hostel	Department Subnet Gateway	Medium	Found in 1 traces. Example target: chemical.iitd.ac.in
2039	202AC810582-202AC810300	10.10.17.2	4.1 of 29	6.13 Sarpura Hostel	Department Subnet Gateway	Medium	Found in 1 traces. Example target: design.iitd.ac.in
2040	202AC810582-202AC810300	10.10.211.211	3.1 of 29	7.51 Sarpura Hostel	Department Subnet Gateway	Medium	Found in 1 traces. Example target: textile.iitd.ac.in
2041	202AC810582-202AC810300	10.10.211.216	3.1 of 29	4.96 Sarpura Hostel	Department Subnet Gateway	Medium	Found in 1 traces. Example target: maths.iitd.ac.in
2042	202AC810582-202AC810300	10.255.109.100	2.15 of 29	5.01 Sarpura Hostel	Campus Core Distribution	High	Found in 15 traces. Example target: www.whatsapp.com
2043	202AC810582-202AC810300	10.255.107.3	2.16 of 29	5.20 Sarpura Hostel	Campus Core Distribution	High	Found in 16 traces. Example target: www.instagram.com
2044	202AC810582-202AC810300	10.119.233.65	4.16 of 29	5.09 Sarpura Hostel	Internal Border Transit	High	Found in 16 traces. Example target: www.facebook.com
2045	202AC810582-202AC810300	10.1.207.69	5.6 of 29	29.22 Sarpura Hostel	NNN Peering Transit	High	Found in 6 traces. Example target: www.google.com
2046	202AC810582-202AC810300	10.1.208.137	6.9 of 29	34.26 Sarpura Hostel	NNN Peering Transit	High	Found in 6 traces. Example target: www.google.com
2047	202AC810582-202AC810300	10.255.238.122	7.1 of 29	46.33 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: 8.8.8.8
2048	202AC810582-202AC810300	10.152.7.214	8.3 of 29	44.6 Sarpura Hostel	Campus Transit Router	High	Found in 3 traces. Example target: www.whatsapp.com
2049	202AC810582-202AC810300	10.1.207.122	10.2 of 29	14.49 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.iitb.ac.in
2050	202AC810582-202AC810300	10.120.123.118	7.1 of 29	14.32 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.iitb.ac.in
2051	202AC810582-202AC810300	10.1.207.121	8.1 of 29	43.07 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.iitb.ac.in
2052	202AC810582-202AC810300	10.255.238.254	10.2 of 29	44.47 Sarpura Hostel	Campus Transit Router	High	Found in 2 traces. Example target: www.iitb.ac.in
2053	202AC810582-202AC810300	10.119.249.49	11.1 of 29	42.98 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.iitb.ac.in
2054	202AC810582-202AC810300	10.119.249.50	12.1 of 29	42.96 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.iitb.ac.in
2055	202AC810582-202AC810300	10.10.10.1	13.1 of 29	51.74 Sarpura Hostel	Department Subnet Gateway	Medium	Found in 1 traces. Example target: www.iitb.ac.in
2056	202AC810582-202AC810300	10.119.234.162	6.9 of 29	8.55 Sarpura Hostel	Commercial Egress Gateway	High	Found in 1 traces. Example target: www.amazon.com
2057	202AC810582-202AC810300	10.255.239.30	7.1 of 29	12.77 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.instagram.com
2058	202AC810582-202AC810300	10.255.222.229	6.1 of 29	6.11 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.instagram.com
2059	202AC810582-202AC810300	10.255.222.1	8.1 of 29	12.72 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.instagram.com
2060	202AC810582-202AC810300	10.24.250.102	12.91 of 29	12.91 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.facebook.com
2061	202AC810582-202AC810300	10.120.71.174	8.1 of 29	12.89 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.facebook.com
2062	202AC810582-202AC810300	10.255.221.1	8.1 of 29	12.83 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.facebook.com
2063	202AC810582-202AC810300	10.255.237.94	7.1 of 29	48.58 Sarpura Hostel	Campus Transit Router	Medium	Found in 1 traces. Example target: www.whatsapp.com

Figure 4.3: Our internal-hop rows in COL334_A1_Campus_Map.xlsx.

2656	2024CS10582-2024CS10300	8.8.8.8	11 1 of 28	63.75 Satpura Hostel	Google DNS Server (Destination)	Medium	19-08-2026	Found in 1 traces. Target: 8.8.8.8
2657	2024CS10582-2024CS10300	50.144.234.93	7 5 of 28	22.86 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 3 traces. Example target: www.microsoft.com
2658	2024CS10582-2024CS10300	116.119.42.218	2 2 of 28	88.23 Satpura Hostel	CDN Edge / International Transit	High	19-08-2026	Found in 2 traces. Example target: www.mit.edu
2659	2024CS10582-2024CS10300	23.54.79.22	10 1 of 28	86.68 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.mit.edu
2660	2024CS10582-2024CS10300	23.54.79.11	11 1 of 28	87.22 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.mit.edu
2661	2024CS10582-2024CS10300	23.54.79.4	12 1 of 28	75.47 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.mit.edu
2662	2024CS10582-2024CS10300	23.54.118.193	13 1 of 28	86.31 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.google.com
2663	2024CS10582-2024CS10300	72.14.204.62	9 1 of 28	60.03 Satpura Hostel	External Transit / ISP Node	Medium	19-08-2026	Found in 1 traces. Example target: www.google.com
2664	2024CS10582-2024CS10300	142.251.151.119	10 1 of 28	71.38 Satpura Hostel	External Transit / ISP Node	Medium	19-08-2026	Found in 1 traces. Example target: www.instagram.com
2665	2024CS10582-2024CS10300	157.240.68.238	9 1 of 28	58.32 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.instagram.com
2666	2024CS10582-2024CS10300	129.134.24.128	10 1 of 28	57.13 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.instagram.com
2667	2024CS10582-2024CS10300	129.134.84.43	11 1 of 28	72.39 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.instagram.com
2668	2024CS10582-2024CS10300	157.240.65.230	9 2 of 28	131.77 Satpura Hostel	Meta Peering / Transit Edge	High	19-08-2026	Found in 2 traces. Example target: www.whatsapp.com
2669	2024CS10582-2024CS10300	129.134.47.42	10 1 of 28	120.24 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.facebook.com
2670	2024CS10582-2024CS10300	129.134.89.205	11 1 of 28	117.06 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.facebook.com
2671	2024CS10582-2024CS10300	116.119.121.117	8 1 of 28	70.01 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.whatsapp.com
2672	2024CS10582-2024CS10300	129.134.46.119	10 1 of 28	130.58 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.whatsapp.com
2673	2024CS10582-2024CS10300	129.134.85.164	11 1 of 28	117.63 Satpura Hostel	Meta Peering / Transit Edge	Medium	19-08-2026	Found in 1 traces. Example target: www.whatsapp.com
2674	2024CS10582-2024CS10300	162.158.226.122	12 1 of 28	74.14 Satpura Hostel	External Transit / ISP Node	Medium	19-08-2026	Found in 1 traces. Example target: www.x.com
2675	2024CS10582-2024CS10300	162.158.226.71	13 1 of 28	62.31 Satpura Hostel	External Transit / ISP Node	Medium	19-08-2026	Found in 1 traces. Example target: www.x.com
2676	2024CS10582-2024CS10300	162.159.140.229	14 1 of 28	62.28 Satpura Hostel	External Transit / ISP Node	Medium	19-08-2026	Found in 1 traces. Example target: www.x.com
2677	2024CS10582-2024CS10300	136.232.148.177	7 2 of 28	16.54 Satpura Hostel	External Transit / ISP Node	High	19-08-2026	Found in 2 traces. Example target: www.amazon.com
2678	2024CS10582-2024CS10300	14.143.159.145	7 1 of 28	19.43 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.apple.com
2679	2024CS10582-2024CS10300	14.143.159.206	10 1 of 28	67.51 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.apple.com
2680	2024CS10582-2024CS10300	104.70.119.38	11 1 of 28	70.82 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.apple.com
2681	2024CS10582-2024CS10300	23.65.190.73	12 1 of 28	89.18 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.apple.com
2682	2024CS10582-2024CS10300	104.70.119.37	13 1 of 28	79.22 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.apple.com
2683	2024CS10582-2024CS10300	104.70.119.25	14 1 of 28	73.56 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.apple.com
2684	2024CS10582-2024CS10300	116.119.119.198	8 1 of 28	89.61 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.microsoft.com
2685	2024CS10582-2024CS10300	23.54.79.23	10 1 of 28	87.36 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.microsoft.com
2686	2024CS10582-2024CS10300	23.54.79.19	11 1 of 28	75.41 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.microsoft.com
2687	2024CS10582-2024CS10300	23.54.79.42	12 1 of 28	87.28 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.microsoft.com
2688	2024CS10582-2024CS10300	23.161.164.161	13 1 of 28	73.33 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.microsoft.com
2689	2024CS10582-2024CS10300	23.58.138.79	9 1 of 28	86.8 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.ibm.com
2690	2024CS10582-2024CS10300	23.58.138.19	10 1 of 28	122.56 Satpura Hostel	CDN Edge / International Transit	Medium	19-08-2026	Found in 1 traces. Example target: www.ibm.com

Figure 4.4: Our external transit and CDN-edge rows in COL334_A1_Campus_Map.xlsx.

Supports: Chapter 2, Table 2.1, Figure 2.1, and Section A2.3 (contribution count).

E-5: command output

Who: Tejasvi Shrivastava (2024CS10582) When: 22-08-2026, 16:15 Where: NB-09, Satpura Hostel

Artefact: `ip addr` (confirming interface `wlo1`) and `traceroute -n www.codechef.com`, verbatim:

```
> traceroute -n www.codechef.com
traceroute to www.codechef.com (172.67.75.52), 30 hops max, 60 byte packets
 1  10.184.0.13      2.759 ms  2.676 ms  2.644 ms
 2  10.255.109.100   2.527 ms  2.498 ms  2.472 ms
 3  10.255.107.3     3.784 ms  3.759 ms  4.793 ms
 4  10.119.233.65    3.703 ms  3.679 ms  3.652 ms
 5  * * *
 6  10.119.234.162   4.620 ms  4.465 ms  4.341 ms
 7  14.143.159.145   15.750 ms * 26.497 ms
 8  * * *
 9  * 121.240.255.30 6.970 ms *
10  104.23.231.5     8.611 ms * 104.23.231.30 16.686 ms
11  172.67.75.52     16.622 ms 4.546 ms 14.703 ms
```

(This is a separate run from the one quoted in Section 1.3; hop sequence and roles are identical, RTTs differ run-to-run as expected.)

Supports: Chapter 1, Section “Finding T1”; Figure 1.3 (shared-gateway note).

E-6: command output

Who: Yashvardhan Singh Chaudhary (2024CS10300) When: 22-08-2026, 16:25 Where: NB-01, Satpura Hostel

Artefact: `ipconfig` and `tracert -d www.codechef.com`, run on Windows, verbatim:

```
Wireless LAN adapter Wi-Fi:
```



```

Connection-specific DNS Suffix  . : iitd.ac.in
IPv4 Address. . . . . : 10.184.20.91
Subnet Mask . . . . . : 255.255.224.0
Default Gateway . . . . . : 10.184.0.1

> tracert -d www.codechef.com
Tracing route to www.codechef.com [172.67.75.52]
 0  6 ms    4 ms    2 ms  10.184.0.13
 1  8 ms    8 ms   13 ms  10.255.109.100
 2 15 ms   15 ms   29 ms  10.255.107.3
 3  6 ms    3 ms    2 ms  10.119.233.65
 4  *      *      *      Request timed out.
 5  5 ms    4 ms    5 ms  10.119.234.162
 6 17 ms   16 ms   18 ms  14.143.159.145
 7  *      *      *      Request timed out.
 8 34 ms    *      47 ms  121.240.255.30
 9  *      34 ms   21 ms  104.23.231.30
10  7 ms    7 ms    9 ms  172.67.75.52
Trace complete.

```

This independently confirms two things from a second device on a second OS: the same **10.184.0.1** DHCP gateway / **10.184.0.13** first-hop discrepancy noted in Section 1.2, and the identical hop-5/hop-8 timeout pattern discussed in the diagram’s analytical notes: since both teammates see the same two hops go silent independently, a probe-specific fluke on one device is less likely than a deliberate no-reply configuration on those routers.

Supports: Chapter 1, Section “Finding T1”; Figure 1.3 (shared-gateway note).

E-7: command output / lookup

Who: Tejasvi Shrivastava (2024CS10582) When: 19-08-2026, 19:35 Where: NB-09, Satpura Hostel

Artefact: **whois** lookups for the four public hops in Table 1.1, verbatim:

```

> whois 14.143.159.145 | grep -iE "orgname|netname|descr"
netname:      TATACOMM-IN
descr:        Internet Service Provider
descr:        TATA Communications formerly VSNL is Leading ISP,
descr:        Data and Voice Carrier in India
descr:        Tata Communications Limited

> whois 121.240.255.30 | grep -iE "orgname|netname|descr"
netname:      TATACOMM-IN
descr:        Route for VSNL

> whois 104.23.231.30 | grep -iE "orgname|netname|descr"
NetName:      CLOUDFLARENET

```

```
OrgName:          Cloudflare, Inc.

> whois 172.67.75.52 | grep -iE "orgname|netname|descr"
NetName:          CLOUDFLARENET
OrgName:          Cloudflare, Inc.
```

Supports: Chapter 1, Section A1.3, Table 1.1.

E-8: log

Who: Tejasvi Shrivastava (2024CS10582) When: quiet hour, 20-08-2026, 05:40

Where: NB-09, Satpura Hostel

Artefact: 300-packet continuous ping to T3, quiet hour, first/last 10 lines:

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=112 time=46.4 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=112 time=47.1 ms
... (280 lines omitted; full log in evidences/e7.txt) ...
64 bytes from 8.8.8.8: icmp_seq=299 icmp_seq=299 ttl=112 time=49.3 ms
64 bytes from 8.8.8.8: icmp_seq=300 ttl=112 time=46.9 ms

--- 8.8.8.8 ping statistics ---
300 packets transmitted, 300 received, 0% packet loss, time 299257ms
rtt min/avg/max/mdev = 44.269/48.832/154.554/10.852 ms
```

Supports: Chapter 3, Figure 3.3.

E-9: log

Who: Tejasvi Shrivastava (2024CS10582) When: busy hour, 22-08-2026, 15:11

Where: NB-09, Satpura Hostel

Artefact: 300-packet continuous ping to T3, busy hour, first/last 10 lines:

```
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=114 time=32.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=114 time=27.2 ms
... (280 lines omitted; full log in evidences/e8.txt) ...
64 bytes from 8.8.8.8: icmp_seq=299 ttl=114 time=25.0 ms
64 bytes from 8.8.8.8: icmp_seq=300 ttl=114 time=26.1 ms

--- 8.8.8.8 ping statistics ---
300 packets transmitted, 300 received, 0% packet loss, time 299353ms
rtt min/avg/max/mdev = 24.356/31.706/192.284/17.041 ms
```

Supports: Chapter 3, Figure 3.3.

E-10: command output

Who: Tejasvi Shrivastava (2024CS10582) When: 19-08-2026, 19:35 Where: NB-09,

Satpura Hostel

Artefact: 50-packet ping summaries for T1 and T2:

```
> ping -c 50 -s 64 10.184.0.13
--- 10.184.0.13 ping statistics ---
50 packets transmitted, 50 received, 0% packet loss, time 49072ms
rtt min/avg/max/mdev = 3.904/8.891/96.290/12.787 ms

> ping -c 50 -s 64 www.iitd.ac.in
--- www.iitd.ac.in ping statistics ---
50 packets transmitted, 50 received, 0% packet loss, time 49078ms
rtt min/avg/max/mdev = 3.315/7.489/14.110/2.716 ms
```

Supports: Chapter 3, Table 3.1, Figure 3.1.

E-11: command output

Who: Tejasvi Shrivastava (2024CS10582) When: 20-08-2026, 04:50 Where: NB-09, Satpura Hostel

Artefact: 50-packet ping summaries for T3 and T5 (note the minor packet loss on both, 2% and 4% respectively, not previously called out in Chapter 3):

```
> ping -c 50 -s 64 8.8.8.8
--- 8.8.8.8 ping statistics ---
50 packets transmitted, 49 received, 2% packet loss, time 49119ms
rtt min/avg/max/mdev = 42.824/49.859/119.830/11.032 ms

> ping -c 50 -s 64 www.mit.edu
--- www.mit.edu ping statistics ---
50 packets transmitted, 48 received, 4% packet loss, time 49175ms
rtt min/avg/max/mdev = 35.596/51.145/247.982/36.326 ms
```

Supports: Chapter 3, Table 3.1, Figure 3.1.

E-12: command output

Who: Tejasvi Shrivastava (2024CS10582) When: 20-08-2026, 04:50 Where: NB-09, Satpura Hostel

Artefact: 100% packet loss for T4:

```
> ping -c 50 -s 64 www.iitb.ac.in
PING www.iitb.ac.in (103.21.124.133) 64(92) bytes of data.
--- www.iitb.ac.in ping statistics ---
50 packets transmitted, 0 received, 100% packet loss, time 50162ms
```

Supports: Chapter 3, Table 3.1, Section A3.1 firewall-drop discussion.

E-13: command output

Who: Tejasvi Shrivastava (2024CS10582) When: 19-08-2026, 20:00 Where: NB-09, Satpura Hostel

Artefact: packet-size sweep summaries for T1 (64/256/512/1024/1400 bytes):

```
--- 10.184.0.13 ping statistics --- (size 64)
rtt min/avg/max/mdev = 3.186/10.550/110.213/20.142 ms
--- 10.184.0.13 ping statistics --- (size 256)
rtt min/avg/max/mdev = 2.838/5.688/27.777/4.310 ms
--- 10.184.0.13 ping statistics --- (size 512)
rtt min/avg/max/mdev = 2.891/6.757/44.268/7.115 ms
--- 10.184.0.13 ping statistics --- (size 1024)
rtt min/avg/max/mdev = 3.043/6.273/20.171/3.219 ms
--- 10.184.0.13 ping statistics --- (size 1400)
rtt min/avg/max/mdev = 3.202/6.025/11.310/1.969 ms
```

Supports: Chapter 3, Figure 3.2, T1 regression.

E-14: command output

Who: Tejasvi Shrivastava (2024CS10582) When: 19-08-2026, 20:15 Where: NB-09, Satpura Hostel

Artefact: packet-size sweep summaries for T3 (64/256/512/1024/1400 bytes):

```
--- 8.8.8.8 ping statistics --- (size 64)
rtt min/avg/max/mdev = 44.637/46.964/52.083/1.430 ms
--- 8.8.8.8 ping statistics --- (size 256)
rtt min/avg/max/mdev = 44.587/52.202/183.043/24.330 ms
--- 8.8.8.8 ping statistics --- (size 512)
rtt min/avg/max/mdev = 44.733/50.704/83.341/7.764 ms
--- 8.8.8.8 ping statistics --- (size 1024)
rtt min/avg/max/mdev = 44.709/49.003/84.001/6.643 ms
--- 8.8.8.8 ping statistics --- (size 1400)
rtt min/avg/max/mdev = 45.368/47.889/52.012/1.350 ms
```

Supports: Chapter 3, Figure 3.2, T3 regression.

E-15: command output

Who: Tejasvi Shrivastava (2024CS10582) When: 22-08-2026, 20:43 Where: NB-09, Satpura Hostel

Artefact: independent `tracert -n + ping -c 10` verification of two rows submitted by team 2024CS50261+2024CS51073, both returning full timeouts:

```
5 traceroute -n 10.255.221.1
ping -c 10 10.255.221.1
traceroute to 10.255.221.1 (10.255.221.1), 30 hops max, 60 byte packets
 1 * * *
 2 * * *
 3 * * *
 4 * * *
 5 * * *
 6 * * *
 7 * * *
 8 * * *
 9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
22 * * *
23 * * *
24 * * *
25 * * *
26 * * *
27 * * *
28 * * *
29 * * *
30 * * *
PING 10.255.221.1 (10.255.221.1) 56(84) bytes of data.
--- 10.255.221.1 ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 9226ms
```

Figure 4.5: Row 1092 (10.255.221.1, claimed hop 11): 30/30 traceroute hops and 10/10 pings timed out. Result: couldn't reach.

```
5 traceroute -n 10.255.237.94
ping -c 10 10.255.237.94
traceroute to 10.255.237.94 (10.255.237.94), 30 hops max, 60 byte packets
 1 * * *
 2 * * *
 3 * * *
 4 * * *
 5 * * *
 6 * * *
 7 * * *
 8 * * *
 9 * * *
10 * * *
11 * * *
12 * * *
13 * * *
14 * * *
15 * * *
16 * * *
17 * * *
18 * * *
19 * * *
20 * * *
21 * * *
22 * * *
23 * * *
24 * * *
25 * * *
26 * * *
27 * * *
28 * * *
29 * * *
30 * * *
PING 10.255.237.94 (10.255.237.94) 56(84) bytes of data.
--- 10.255.237.94 ping statistics ---
10 packets transmitted, 0 received, 100% packet loss, time 9230ms
```

Figure 4.6: Row 1096 (10.255.237.94, claimed hop 7): 30/30 traceroute hops and 10/10 pings timed out. Result: couldn't reach.

Supports: Chapter 2, Section A2.3 (verification).

Chapter 5

Three Findings

F-1: Diagnostic Blackholes in Transit

Saw: Hops 5 and 8 in the traceroute to `www.codechef.com` returned 100% timeouts (* * *) on both teammates' independent runs.

Evidence: E-5, E-6

Think: We conclude that these specific transit routers are configured not to generate ICMP time-exceeded replies, obscuring internal infrastructure from standard traceroute-based mapping. One other explanation we couldn't rule out is that these routers have a rate-limited ICMP control plane that happened to be saturated by background traffic on both our probes, dropping our specific packets rather than enforcing a hard no-reply policy, though the fact that *both* teammates saw exactly the same two hops go silent, on different days and devices, makes coincidental rate-limiting the less likely of the two.

Fix: Enable ICMP time-exceeded (Type 11, Code 0) generation on these internal routers, with a conservative rate limit (e.g. 50 pps) so legitimate diagnostics and PM-TUD keep working without opening a full information-disclosure path.

Check: Re-run `traceroute -n www.codechef.com`; hops 5 and 8 should return valid IPs and RTTs consistent with their neighbours, instead of asterisks.

F-2: First-Hop Wi-Fi Delay Floor

Saw: The minimum RTT to the immediate local gateway (10.184.0.13) across 50 probes was 3.904ms, not the sub-millisecond floor a single wired hop would suggest.

Evidence: E-10

Think: We conclude this reflects the Wi-Fi RF environment in the hostel: CSMA/CA contention and backoff on a busy 2.4/5 GHz band, rather than anything downstream of the gateway itself, consistent with our reading of this floor in Section 3.1. One other explanation we couldn't rule out is that the access point's CPU is under load from

managing a large number of simultaneous MBSSID client associations, adding a small software-level queuing delay before a packet even reaches the routing fabric: a device-side bottleneck rather than a wireless-medium one.

Fix: Add access points in the hostel corridors and reduce transmit power on existing ones to shrink cell size and cut co-channel contention, rather than relying on the current handful of APs to cover the whole floor.

Check: Repeat `ping -c 50 10.184.0.13` from the same location; if the RF explanation is right, the minimum RTT should measurably drop once contention is reduced (we do not have a specific target threshold without knowing the current channel load, but a drop toward the sub-1 ms range typical of an uncontended wired first hop would confirm it).

F-3: Transient Congestion Spikes at Busy Hour

Saw: During the busy-hour capture, RTT to T3 sat on a ~ 27 ms baseline but spiked repeatedly to as high as ~ 192 ms, compared to a flat ~ 48 ms baseline with only isolated spikes (max ~ 155 ms) at quiet hour (Chapter 3, Figure 3.3).

Evidence: E-9, Figure 3.3

Think: We conclude the sharp-in/sharp-out spike shape reflects bursty cross-traffic filling a router buffer and draining quickly once the burst ends, rather than sustained congestion; this is consistent with what Chapter 3 already argues from the same plot. One other explanation we couldn't rule out is that these spikes are ISP-side micro-outages or momentary throttling at the Tata Communications peering edge identified in Chapter 1, rather than a purely local buffer filling: we have no way to distinguish "our packet queued behind others" from "the upstream link itself briefly stalled" without visibility into that router, which is outside what we can observe from a single end host.

Fix: If the cause is local buffering, enabling an active queue management scheme (e.g. fq_codel) on the campus egress gateway would bound queuing delay by dropping/marking early instead of letting the buffer fill; this would not help if the cause is upstream, which is why distinguishing the two matters before recommending a fix with confidence.

Check: Repeat the 300-packet busy-hour capture; under the local-buffering explanation, adding AQM should visibly cap spike height in a later capture. Under the upstream explanation, spikes would persist regardless of any change made on campus-controlled equipment, which would itself be informative.

At least one finding above (F-1) comes from the topology work and at least two (F-2, F-3) come from the delay experiment, per the assignment's requirement of at least one from each.

Chapter 6

Two Days, Two Devices

This chapter answers **B1** of Part B (10 marks). Both teammates captured six ten-minute windows each, spread over two days and three time bands, and this chapter compares what those windows show about round-trip time and about what the machines were doing when we were not looking.

Team: 2024CS10300 (Windows 11, MediaTek Wi-Fi 6) and 2024CS10582 (Ubuntu, Intel wlo1). The RTT figures come from the per-window `netlab_gen` logs, 20 ICMP echoes per target per window. All twelve windows are complete.

6.1 Caveats, stated once and relied on throughout

(a) **Address family.** T2 and T5 are not strict cross-device absolute-RTT comparisons. 2024CS10582 has IPv6 and reached `www.iitd.ac.in` at `2001:df4:e000:29::212` and `www.mit.edu` at `2600:140f:5:2093::255e`; 2024CS10300 is IPv4-only and reached `10.10.211.212` and `23.15.150.186`. In D2W3 this produces a 95.0ms vs 8.55ms median split on T5: the same hostname, two different Akamai nodes, and no useful comparison between them. **T3 (8.8.8.8) is IPv4 on both hosts and is the only clean cross-device comparison**, which is why the analysis below leans on it.

(b) **Resolution.** Windows `ping` reports whole milliseconds, Linux two decimals. A Windows median of “48” means 47.5–48.5.

(c) **p95 from 20 samples is the second-largest value.** It moves on one or two packets. We treat large p95 excursions as observations, not as distribution shifts.

(d) **Per-hop traceroute times are unreliable on this path.** See Section [6.4.2](#).

6.2 Data

Table 6.1: T2: `www.iitd.ac.in` (on campus). Median and p95 RTT in ms, 20 echoes per window per device.

Window	Date	10300 median	10300 p95	10582 median	10582 p95
D1W1	22-08	4.0	9.3	2.65	6.85
D1W2	20-08	7.0	7.3	3.83	5.14
D1W3	20-08	8.0	17.6	5.71	12.80
D2W1	21-08	4.0	6.0	5.12	5.84
D2W2	21-08	5.5	11.1	4.46	8.37
D2W3	22-08	5.0	22.20	3.03	5.73

Table 6.2: T3: 8.8.8.8 (off campus). Median and p95 RTT in ms. The two windows recorded on 22-08 are set in bold.

Window	Date	10300 median	10300 p95	10582 median	10582 p95
D1W1	22-08	59.0	66.1	60.50	61.08
D1W2	20-08	48.0	60.1	50.35	54.19
D1W3	20-08	46.0	53.2	45.80	48.27
D2W1	21-08	46.0	52.9	46.25	51.28
D2W2	21-08	45.0	49.0	44.60	48.11
D2W3	22-08	65.0	69.05	59.70	83.09

T4 (`www.iitb.ac.in`, 103.21.124.133) returned 0 of 20 echo replies in every window on both devices, so it carries no RTT series, the same 100% loss Part A reports in Chapter 3. Its traceroute consistently reached 10.10.10.1 before subsequent probes timed out. This shows probes progressed a long way down the path but does not establish that the destination host was reachable: a router can answer TTL-expired probes while the destination is unreachable or filters echo. Every hop on the T4 path is RFC1918 (10.184.0.13 ... 10.10.10.1, thirteen hops), whereas the T3 path shows public addresses from hop 9 onward; on the T4 path we never observe a packet in public address space at all.

6.3 Plots

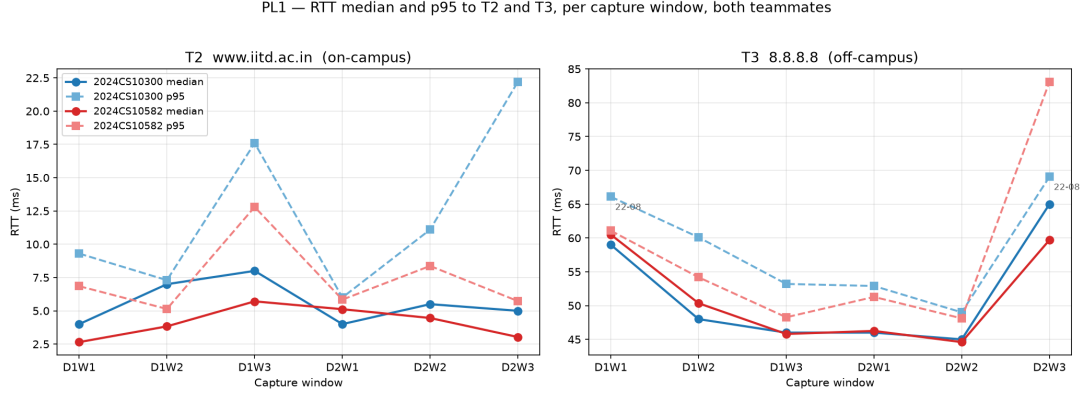


Figure 6.1: **PL1.** T3 RTT separates cleanly by calendar date, not by time of day: both windows recorded on 22-08 (D1W1, D2W3) sit at 59–65 ms on both devices, while all four windows recorded on 20-08 and 21-08 sit at 44.6–50.4 ms. Two independent laptops moving together by ~ 14 ms across a date boundary points at the shared upstream path rather than at either machine. The T2 series shows no comparable structure.

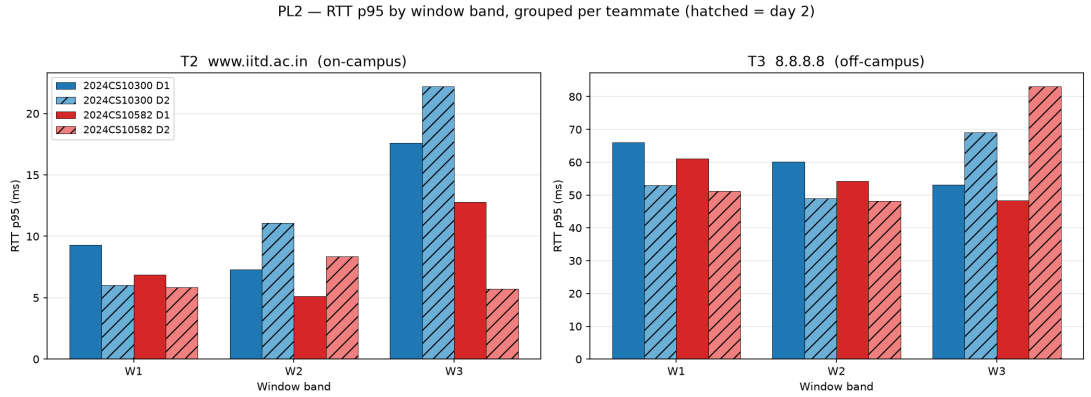


Figure 6.2: **PL2.** T2 p95 is largest in W3 for both teammates (22.20 ms and 12.80 ms), but the two devices peak on different days and disagree by a factor of four within the same evening, so the evening effect on the campus path is weak and inconsistent. T3 p95 shows no band structure at all; its variation follows the date, which is visible in PL1 and invisible here.

6.4 Q1: Which window was worst for each of us? Same or different?

6.4.1 T3 (off campus): the same two windows are worst for both of us, and they share a date

D2W3 is the worst T3 window for both teammates (65.0 ms median for 2024CS10300 and 59.70 ms for 2024CS10582), and D1W1 is second worst for both, at 59.0 ms and 60.50 ms. Every other window sits between 44.6 and 50.4 ms on both machines.

The two elevated windows are D1W1 (22-08, 09:00 IST) and D2W3 (22-08, 21:00 IST). The four normal windows are 20-08 and 21-08. **The split is by date, not by hour**, and the data contains its own control: D2W1 was recorded on 21-08 at 09:20 and D1W1 on 22-08 at 09:00, twenty minutes apart on the clock, one day apart on the calendar, and 13 ms apart in median RTT (46.0 vs 59.0 on this device; 46.25 vs 60.50 on the other). Whatever changed did not care what time it was. Time-of-day congestion cannot produce that pattern.

What moved is the floor, not the spread. 2024CS10582's D1W1 ping reports min 59.7 / avg 60.5 / max 62.6 ms with mdev 0.578 ms, the tightest distribution in our whole set. Congestion produces a low minimum with a long upper tail; here the minimum itself is displaced and the tail is nearly absent. This is more consistent with a change of path than with ordinary queueing.

Two explanations we cannot separate. 8.8.8.8 is anycast, so a different Google front-end selected on 22-08 would produce the same uniform floor shift as an upstream re-route by our own provider. Hop 9 does vary across our traces (72.14.204.62 in some windows, 142.250.172.80 in others), which is consistent with either. Ping and traceroute alone cannot decide it. What we can say is that whatever it was persisted for at least twelve hours and was visible from two machines on different operating systems.

6.4.2 How far can we localise it? Not far, and we can show why

Router 10.1.207.69 appears at hop 5 in both our T3 and T4 traces, in every window, on both machines. In the T3 trace it reports 40–64 ms; in the T4 trace, run minutes later from the same host, it reports 3.6–10 ms.

Table 6.3: The same router, timed twice, minutes apart. Its reported time in the T3 trace tracks the T3 end-to-end RTT rather than any distance to that router.

	T3 trace hop 5	T4 trace hop 5	T3 end-to-end
10300 D2W1	41 ms	6 ms	45 ms
10300 D2W3	58 ms	7 ms	65 ms
10582 D2W1	40.5 ms	8.4 ms	39.5 ms
10582 D2W3	62.2 ms	10.2 ms	61.9 ms

The same router cannot be both 6 ms and 60 ms away. Reading “the jump happens at hop 5” out of these traces would only restate the end-to-end measurement. We suspect tunnelling or ICMP de-prioritisation at that device but cannot demonstrate either.

What the traceroutes do support is a negative result. Hops 1–4 (10.184.0.13, 10.255.109.100, 10.255.107.3, 10.119.233.65) measure 2–7 ms on both devices in the two elevated windows, no different from the normal ones. The added ~ 14 ms is therefore not in our Wi-Fi, not at the gateway, and not in the first three campus routers. **The bottleneck for T3 was far from us**, not something we could have caused or fixed from where we sat.

6.4.3 T2 (on campus): a weak evening effect, and the two devices disagree

The worst T2 window for each of us falls in the W3 band (D2W3 for 2024CS10300, p95 22.20 ms, and D1W3 for 2024CS10582, p95 12.80 ms), and for 2024CS10300 the top two windows are both W3. That is the extent of the agreement.

Against it: in D2W3, recorded 79 seconds apart on the same evening, 2024CS10300 measured a p95 of 22.20 ms and 2024CS10582 measured 5.73 ms, a factor of four on the same network at the same moment. And 2024CS10300’s 22.20 ms rests on a single 64 ms reply out of twenty. With 20 samples the p95 *is* the second-largest value, so one delayed packet moves it several milliseconds.

We report this as a weak and inconsistent effect rather than a time-of-day law. The medians tell the sober version: T2 medians span 2.65–8.0 ms across all twelve windows with no visible trend. Whatever produces the occasional 20–60 ms reply on the campus path is rare, brief, and not shared between two machines sitting on the same network at the same time.

6.4.4 What the two targets together let us say

The one solid result is that a large, sustained, device-independent RTT change appeared on the off-campus path on 22-08 and was invisible on the on-campus path. The one solid non-result is that we could not demonstrate an evening congestion effect: the on-campus tails are small in absolute terms, driven by individual packets, and inconsistent between two devices measuring simultaneously. “No difference” is a legitimate result here and we report it as one.

One measurement we did not think to take. On the last evening we recorded our wireless association: 2024CS10300 was on **IITD_WIFI**, BSSID **e4:4e:2d:0c:82:6f**, 5 GHz; 2024CS10582 was on **eduroam**, BSSID **E4:4E:2D:0C:82:61**, 2.4 GHz. Those are two radios of the *same physical access point* (the BSSIDs share the **E4:4E:2D:0C:82** prefix) carrying different SSIDs on different bands. So in D2W3 our packets converged one hop in and diverged only over the air, and the fourfold T2 p95 difference between us must have arisen in the wireless link or the host, not in the campus network beyond it. The AP itself reported 15 connected stations and 7% channel utilisation at that moment, which does not look like a congested radio. We did not record the association for the earlier five windows, so we cannot extend this reasoning backwards; that is the measurement we would add if we ran this again.

6.5 Q2: Biggest non-workload consumer of bytes on each device

Workload means what the assignment generated: `netlab_gen` traffic to 168.144.154.92 and the T1–T5 pings and traceroutes. Everything else is traffic we did not ask for.

6.5.1 Device 1: 2024CS10300 (Windows 11)

Table 6.4: Per-window totals and the largest non-workload consumer, 2024CS10300.

Window	Total	Largest non-workload consumer
D1W1	190 kB	<code>claude.ai</code> / <code>a-api.anthropic.com</code> 60.5 kB; Office background services
D1W2	529 kB	Datadog RUM beacon <code>browser-intake-us5-datadoghq.com</code> 212.7 kB (~40%)
D1W3	494 kB	Autodesk licensing/CDN $\approx$ 185 kB
D2W1	2 487 kB	<code>api.genuine.autodesk.com</code> 1 414 kB (57%)
D2W2	24 MB	Windows Delivery Optimization $\approx$ 22.9 MB (92%)
D2W3	321 kB	Brave browser updaters: <code>brave-core-ext.s3.brave.com</code> 60.5 kB + <code>go-updater.brave.com</code> 28.8 kB $\approx$ 89 kB (28%)

Named answer: Windows Delivery Optimization. In D2W2 it accounts for roughly 22.9MB of a 24.8MB capture, arriving two ways at once: 15.7MB pulled peer-to-peer over TCP 7680 from four other machines on the campus network (10.152.241.114, 9.3 MB; 10.152.241.24, 2.1 MB; 10.152.200.120, 2.1 MB; 10.152.234.96, 2.1 MB) and 7.2 MB over plain HTTP from 206.206.82.204, whose requests carry `User-Agent: Microsoft-Delivery-Optimization/10.1` and `cacheHostOrigin=dl.delivery.mp.microsoft.com`.

So the answer to “what is your laptop doing when you’re not looking” is: downloading a Windows update from four other machines in this building, and serving them in return, on a port we had never heard of before this assignment. The quiet windows say the same thing more softly: Autodesk licence checks, Brave updaters, Datadog analytics beacons fired by a page that was merely open, Google FCM on 5228, Meta XMPP on 5222.

```
tcp.port == 7680 || http.user_agent contains "Delivery-Optimization"
```

6.5.2 Device 2: 2024CS10582 (Ubuntu)

Table 6.5: Per-window totals and the largest non-workload consumer, 2024CS10582.

Window	Total	Largest non-workload consumer
D1W1	2 947 kB	<code>mobile.events.data.microsoft.com</code> 267 kB, top consumer of the window
D1W2	3 444 kB	MS telemetry 479 kB
D1W3	36 MB	<code>docs.google.com</code> 30.5 MB, see note
D2W1	2 047 kB	MS telemetry, background Google
D2W2	5 678 kB	MS telemetry 696 kB across four endpoints
D2W3	29 MB	<code>excel-telemetry.officeapps.live.com</code> 15.3 MB + <code>inc-excel.officeapps.live.com</code> / <code>common.online.office.com</code> 6.4 MB $\approx$ 21.7 MB (75%)

Named answer: Microsoft Office telemetry and sync. It is the only non-workload consumer present in *every* window on this device, and in D2W3 it is overwhelming: 21.7 MB of a 29 MB capture, with `mobile.events.data.microsoft.com` alone accounting for 92 of the TLS handshakes in that window. In the quiet windows it is a 267–696 kB trickle; in D2W3 it is three quarters of everything the machine sent or received. The same service, two orders of magnitude apart.

The ambiguous case, reported rather than tidied away: the largest single flow on this device across the whole study is 30.5 MB to `docs.google.com` in D1W3, 84% of that window. We cannot tell from the capture whether that was a deliberate upload or

Drive sync running by itself: the payload is TLS and the SNI is all we get. We name it and mark it undecided rather than claiming it as background.

6.5.3 Cross-device comparison

The two background profiles barely overlap. The Windows machine's is dominated by one large, bursty, peer-to-peer update event that touched four campus hosts; the Linux machine's is Microsoft Office telemetry that ranges from a trickle to 15 MB in ten minutes. The Windows box produced the only traffic in our dataset that contacted other student machines directly, worth noting given the assignment's own rule against scanning student devices. Delivery Optimization does it by design, continuously, and neither of us knew it was running.

Chapter 7

The Hunt

This chapter answers **B2** of Part B (17 marks): finding the planted anomalies in our captures, saying how many there were, and giving a display filter for each that returns exactly the frames we claim.

Reference window: D2_W2 (21-08-2026, 14:55–15:08 IST). We use one fixed window for both anomaly tables because it is the window in which both teammates hold complete, non-zero captures taken three minutes apart, which makes frame-level comparison between the two devices meaningful. Every filter below was run in **tshark** against the submitted **pcapng** files; the “Frames returned” column is the count that filter actually produces on that file, not an estimate.

7.1 B2.1: How many were planted?

The handout says between three and six, and that working out how many is part of the job. We did not assume the two seeds matched, and they do not: **2024CS10300** has four, **2024CS10582** has six.

We tested all six flavours listed in the handout against both captures and recorded the negatives as well as the positives, because “not in this seed” is a result.

Table 7.1: The six handout flavours tested against both captures. Two are genuinely absent from 2024CS10300’s seed; the evidence for those absences is given below.

Flavour (handout wording)	2024CS10300	2024CS10582
Something repeating on a fixed schedule	found: UDP 20020, 11 s	found: UDP 20023, 9 s
A hostname far longer and stranger than any real one	found: token NVAAAR56	found: token 2GEYWDD
A password sitting in the clear	found: mfarooq24	found: avinash36
A rapid burst of tiny packets across many port numbers	found: 24 ports	found: 48 ports
A packet whose TTL is nothing like its neighbours	absent	found: TTL 17
Traffic using one protocol’s port number for something else	absent	found: HTTP on 10443

7.1.1 Evidence for the two absences on 2024CS10300

Odd TTL: absent. Every packet this host sent to 168.144.154.92 carries `ip.ttl == 128`, the Windows default. The filter `ip.dst == 168.144.154.92 && ip.ttl != 128 && !icmp` returns **0 frames** in all five windows. The `!icmp` term is doing real work: without it the same filter returns 10, because Wireshark also matches the IP header quoted inside the server’s ICMP port-unreachable replies, where the TTL has decremented to 116 in transit. That is an artefact of how the filter is written, not a planted anomaly.

Wrong-port protocol: absent. The complete set of destination ports this host used toward the netlab server is 5763–5786, 8080 and 20020. Port 8080 is a conventional HTTP alternate port carrying HTTP, so nothing here is a protocol/port mismatch. Contrast anomaly B6 below, where plaintext HTTP appears on 10443.

7.2 B2.2: Anomaly table, 2024CS10300 (SEED_TAG 5A11D1FB)

Table 7.2: Four planted anomalies on 2024CS10300. All rows refer to 2024CS10300_D2_W2.pcapng.

#	What it is	Frame	Value it encodes	Display filter	Frames
A1	UDP beacon to the netlab server on a fixed schedule, 95-byte frames, each answered with ICMP port-unreachable	8543	Interval = 11 s (10 sends; deltas 10.907–10.946 s)	<code>udp.dstport == 20020 && ip.dst == 168.144.154.92 && !icmp</code>	10
A2	DNS A queries for eight sequential nonsense hostnames under a lab subdomain	8603	TOKEN = NVAAAR56 , base32 of the label <code>jzlecqkbki2tm</code>	<code>dns.flags.response == 0 && dns.qry.name contains ".probe.col334.lab"</code>	8
A3	HTTP POST carrying credentials in the clear over port 8080	8975	<code>username=mfarooq24</code> <code>password=zermxjpc6g</code> <code>remember=1</code>	<code>http.request.method == "POST" && http.request.uri == "/portal/login"</code>	1
A4	Burst of 24 tiny SYNs to consecutive ports, ~313 ms apart, every one answered RST/ACK	8628	Ports 5763–5786 , 66-byte SYNs, no port open	<code>tcp.flags.syn == 1 && tcp.flags.ack == 0 && ip.dst == 168.144.154.92 && tcp.dstport >= 5763 && tcp.dstport <= 5786</code>	24

7.3 B2.3: Anomaly table, 2024CS10582 (SEED_TAG 5E857C61)

Table 7.3: Six planted anomalies on 2024CS10582. All rows refer to 2024CS10582_D2_W2.pcapng.

#	What it is	Frame	Value it encodes	Display filter	Frames
B1	UDP beacon on a fixed schedule, 113-byte frames	3506	Interval = 9 s (10 sends; deltas 9.000 s)	<code>udp.dstport == 20023 && ip.dst == 168.144.154.92 && !icmp</code>	10
B2	DNS queries for eight nonsense hostnames, asked A and AAAA, bare and with the <code>iiitd.ac.in</code> search suffix	4249	TOKEN = 2GEYWNDD , base32 of the label <code>gjdukwxjzcei</code>	<code>dns.flags.response == 0 && dns.qry.name contains ".probe.col334.lab"</code>	32
B3	HTTP POST carrying credentials in the clear over port 8080	4600	<code>username=avinash36 password=bdrhnygavp remember=1</code>	<code>http.request.method == "POST" && http.request.uri == "/portal/login"</code>	1
B4	Burst of 48 tiny SYNs to consecutive ports, ~50 ms apart	4401	Ports 5370–5417 , 74-byte SYNs	<code>tcp.flags.syn == 1 && tcp.flags.ack == 0 && ip.dst == 168.144.154.92 && tcp.dstport >= 5370 && tcp.dstport <= 5417</code>	48
B5	Seven 51-byte UDP datagrams sent with TTL 17 while every other packet this host sends leaves at TTL 64	4564	TTL = 17 , dst port 30021, ~0.8 s apart	<code>ip.dst == 168.144.154.92 && ip.ttl == 17 && udp.dstport == 30021</code>	7
B6	Plaintext HTTP GET <code>/status</code> on TCP 10443, a port commonly used for HTTPS-alt services	4609	Cleartext HTTP on an HTTPS-alt port; server replies <code>Apache/2.4.58 (Ubuntu)</code>	<code>tcp.dstport == 10443 && http.request.method == "GET"</code>	5

7.4 B2.4: Why these filters and not looser ones

A1 / B1: the `!icmp` term is necessary here. Each beacon datagram is answered by an ICMP port-unreachable that quotes the original UDP header, and Wireshark matches `udp.dstport` inside that quote. Without `!icmp` the filter returns 20 rather than 10, and the count stops meaning “beacons sent”. Verified on 2024CS10300_D2_W2: 20 without, 10 with. We use `ip.dst ==` rather than `ip.addr ==` for the same reason: direction matters to what the count means.

A4 / B4: the `!icmp` term is *not* necessary here, and we checked. The obvious worry is that the beacon problem repeats for the SYN burst. It does not, because the netlab server answers these SYNs at layer 4, not with an ICMP error: frame 8629 in 2024CS10300_D2_W2.pcapng is TCP 5763 → 61994 with the RST and ACK flags set. No ICMP is generated anywhere in the burst, so there is no quoted TCP header to contaminate the count. Running both forms confirms it: 24 with and without `!icmp` for A4, 48 with and without for B4. We left the redundant term out.

What the filter *does* need is the port range. `ip.dst == 168.144.154.92` alone would sweep in the beacon, the POST and (on 2024CS10582) the 10443 traffic. Constraining to SYN-without-ACK plus the observed range isolates the burst exactly and makes the returned count equal the number of ports probed.

B5: scoping the TTL, and a detail worth knowing. `ip.ttl == 17` alone is not specific: the traceroutes in the same capture emit probes at every TTL from 1 to 20, so hop-17 probes match too. Adding `ip.dst == 168.144.154.92 && udp.dstport == 30021` separates the planted flow from our own measurement traffic. This is a case where the obvious filter from the handout’s starter list gives a correct-looking but contaminated answer.

`!icmp` is again unnecessary, and the reason is instructive. The ICMP port-unreachables do exist (frames 4565, 4567 and 4569 answer frames 4564, 4566 and 4568), but the IP header quoted inside them carries `ip.ttl == 5`, not 17. That is the TTL as the datagram *arrived at the server*: sent at 17, arrived at 5, so it crossed 12 hops and still reached the destination with margin to spare. So `ip.ttl == 17` cannot match the quote, and the filter returns 7 either way. The anomaly is a deliberately low initial TTL, not a packet dying in transit.

A2 / B2: the handout’s starter filter misses ours. `dns.qry.name.len > 40` returns 0 planted frames on 2024CS10300. Windows queried the bare name `jzlecqkbki2tm-00.probe.col334.lab`, 33 characters, because the host did not append its `iitd.ac.in` DNS suffix to these lookups. 2024CS10582’s Ubuntu host did append it, producing 44-character names the starter filter catches. The anomaly is not invisible on Windows (our filter finds all eight), but the *suggested length filter* behaves differently on the two machines purely because of resolver configuration. Had we trusted it we would have reported three anomalies on this device and never recovered the token.

We match on `.probe.col334.lab` rather than on the server IP or on `probe`, so the filter picks up the planted lookups and nothing else, and `dns.flags.response == 0` drops the replies so the count equals queries emitted.

B6: a note on dissector dependence. Port 10443 is not in Wireshark’s default HTTP port list, so `http.request.method` resolves here only because Wireshark’s HTTP heuristic dissector is enabled by default. If it is disabled, the filter parses but returns 0. A dissector-independent equivalent, verified to return the same 5 frames, is:

```
tcp.dstport == 10443 && tcp.payload[0:3] == 47:45:54
```

(47 45 54 is GET in ASCII.) We give the readable filter as primary and this as a fallback.

7.5 B2.5: Frames returned is a property of the capture, not of the plant

The planted signatures recur across every window we hold, but the frame counts are not identical everywhere, and the one exception is worth reporting rather than smoothing over.

Filter A2 returns **10 frames in 2024CS10300_D1_W2** and 8 in every other window. The extra two are not extra plants:

Table 7.4: Four frames, two lookups: the Windows resolver failing over to the secondary server.

Frame	t (s)	Sent to	DNS txn ID	Query
1375	422.859	10.7.172.202	0xda45	jzlecqkbki2tm-03.probe.col334.lab
1376	422.884	10.10.1.4	0xda45	jzlecqkbki2tm-03.probe.col334.lab
1383	427.440	10.7.172.202	0x3dc1	jzlecqkbki2tm-06.probe.col334.lab
1384	427.465	10.10.1.4	0x3dc1	jzlecqkbki2tm-06.probe.col334.lab

Same transaction IDs, 25 ms apart, re-sent to the **secondary** resolver. The primary (10.7.172.202) did not answer quickly enough and the Windows resolver failed over to 10.10.1.4. Eight planted lookups, ten query packets.

This is the second time this device’s resolver has changed what the anomaly looks like on the wire, the first being the missing search suffix in Section B2.4. Neither difference is in the plant; both are in the host. It is a reminder that a frame count is a measurement of our own machine as much as of the thing we are hunting.

7.6 B2.6: Required per-teammate block

```
ROLL:      2024CS10300
SEED_TAG:  5A11D1FB
TOKEN:     NVAAAR56
```

```
INTERVAL: 11

ROLL:      2024CS10582
SEED_TAG:  5E857C61
TOKEN:     2GEYWDD
INTERVAL:  9
```

How the tokens decode. The hostname label is base32 with the padding stripped and the case lowered. Restore it: upper-case the label, append = until the length is a multiple of 8, then base32-decode. All eight planted queries carry the same encoded label, so the token can be recovered from any one of them; we verified the label is byte-identical across all eight probes and across every window.

```
import base64
base64.b32decode("JZLECQKBKI2TM" + "====") # -> b'NVAAAR56' (2024CS10300)
base64.b32decode("GJDUKWXXJZCEI" + "====") # -> b'2GEYWDD' (2024CS10582)
```

7.7 B2.7: Cross-device observation

The two seeds share four flavours: periodic UDP beacon, long DNS hostname, cleartext HTTP POST and rapid SYN burst. The TTL anomaly and the wrong-port HTTP service occur only on 2024CS10582. Where the four shared flavours overlap, the seeded parameters differ between the two rolls: beacon port and interval, token, credentials, and the position, width and spacing of the port burst.

Running both tables side by side is what let us be confident that “absent” on 2024CS10300 means absent rather than missed. We knew exactly what a TTL anomaly and a wrong-port service look like in this dataset, because the other device had one of each, so we could search for that specific signature and record its absence with a filter and a count rather than a shrug.

7.8 B2.8: Confirmation in the re-recorded D2W3

Both teammates re-recorded D2W3 on 22-08 (2024CS10300 20:57:38–21:07:38 IST, 2024CS10582 20:58:57–21:08:58 IST, 79 seconds apart, both a full 600.2 s by the pcapng Interface Statistics Block). Every filter in the two tables above was re-run against those files and returns exactly the counts stated:

Table 7.5: Every filter re-run against the re-recorded D2W3 captures. Counts match the reference window exactly.

File	A1/B1	A2/B2	A3/B3	A4/B4	B5	B6	negative TTL check
2024CS10300_D2.W3	10	8	1	24	—	—	0
2024CS10582_D2.W3	10	32	1	48	7	5	—

The D1W2 resolver failover described in Section B2.5 does not recur here: A2 returns 8, the plant count, so the primary resolver answered every query in this window.

Chapter 8

Argue Against Yourself

This chapter answers **B3** of Part B (8 marks): which anomaly would be hardest to find at scale, what the innocent explanation for it would be, and what a network operator can see that we cannot.

8.1 Hardest to spot at a million times this traffic: the 11-second beacon (A1)

Not because it hides, but because its harmless twin is everywhere. In a single ten-minute capture (2024CS10300_D2_W2), this one laptop contacted 63 distinct remote addresses and issued 65 DNS queries. Much of that traffic is small, periodic and aimed at a fixed port: resolver queries to 10.7.172.202, QUIC keepalives, Google FCM on 5228, Meta XMPP on 5222, Delivery Optimization peer chatter. The planted beacon is 10 datagrams of 95 bytes on UDP 20020 at 11.0-second intervals. Scaled by a million, a detector tuned to “small UDP datagram, fixed destination port, regular interval” would return essentially every keepalive on the network, and the beacon would land somewhere in the middle of that pile, ranked by nothing.

Contrast the SYN burst: 24 SYNs walking consecutive ports in eight seconds is rare enough that a coarse rule catches it and the false-positive rate stays survivable. Periodicity is not a scarce signal; consecutive-port scanning is. That difference in base rate, not stealth, makes A1 the needle and A4 the haystack’s only interesting straw.

8.2 The innocent case for the beacon

Every one of those datagrams is answered with an ICMP port-unreachable: nothing is listening. The obvious benign reading is a stale endpoint. An

application configured against a collector that has since been decommissioned (a telemetry agent, a licence heartbeat, a monitoring client) will keep firing on its timer indefinitely, because UDP gives it no feedback it is obliged to act on. Our own captures contain the genus if not the species: 2024CS10582 sent 21.7MB to `excel-telemetry.officeapps.live.com` in one ten-minute window, and 2024CS10300 runs `clientstatesvc.delivery.autodesk.com` and `go-updater.brave.com` on schedules neither of us knowingly started. A beacon to a dead port is what that family of software looks like after someone retires a server and forgets a client.

Two things in our capture separate the readings. First, the timing is too good: the ten inter-packet gaps span 10.907–10.946s, a spread of 39ms over 100 seconds. Application timers drift with scheduler jitter and system load; this one does not. Second, and more telling, the destination was never resolved. Filtering `dns.a == 168.144.154.92` across all twelve captures returns zero frames: the address is hardcoded, not looked up. Legitimate telemetry clients almost always reach their collector through a name, because that is how operators move and retire them. A hardcoded literal that never backs off despite being rejected ten times running is doing something a maintained application would not.

8.3 One thing an operator sees that we cannot, and one we see that they cannot

An operator has base rates. From flow records across campus they could ask how many other machines beacon to 168.144.154.92 on UDP 20020, and how many talk to that host at all: the question we cannot answer from one laptop, and the one that decides whether the beacon is a lab plant or an infection. They would also have known our wireless association throughout. We discovered only on the final evening that we were on different SSIDs (`IITD_WIFI` and `eduroam`) served by two radios of the *same* access point, `E4:4E:2D:0C:82`; for the previous five windows we simply do not know, and that gap limits what we could conclude about the on-campus tail latency in Chapter 6.

We have payload. `username=mfarooq24&password=zermxjpc6g` is legible in frame 8975 because we hold the bytes; an operator watching aggregated flow records sees a short HTTP session to port 8080. The same asymmetry recovered the base32 token from the DNS labels. Breadth versus depth: they see the population, we see the contents.